CIE

COPYRIGHT INFORMATION EXTRACTION FORM

TITLE: QuickShow – Smart Movie Show Booking and Seat Reservation System

SCHOOL: Chitkara University, Rajpura, Punjab

DETAIL OF THE STUDENT WHO UPLOADED THE COPYRIGHT ON GOVT PORTAL NEED TO SHARE LOGIN AND PASSWORD WITH MENTORS

Name: Sanchit

Saini Roll No:

2211981342

ALL MEMBERS DETAILS INCLUDING SUPERVISOR

NAME	EMP CODE/ROLL	ADDRESS WITH EMAIL AND MOBILE NO.
Sanchit Saini	2211981342	Chitkara University, Patiala National Highway, Distt. Patiala, Punjab, India -140401 sanchit1342.be22@chitkarauniversity.edu.in (9465465606)
Shivanshu Garg	2211981375	Chitkara University, Patiala National Highway, Distt. Patiala, Punjab, India -140401 shivanshu1375.be22@chitkarauniversity.edu.in (7807200645)
Rohit	2211981316	Chitkara University, Patiala National Highway, Distt. Patiala, Punjab, India -140401 rohit1316.be22@chitkarauniversity.edu.in (8894425395)
Shivani	2211981370	Chitkara University, Patiala National Highway, Distt. Patiala, Punjab, India -140401 shivani1370.be22@chitkarauniversity.edu.in (7973004232)
Dr. Swati Goel		Chitkara University, Patiala National Highway, Distt. Patiala, Punjab, India -140401 swati.goel@chitkara.edu.in (9530201003)

Department of Computer Science and Engineering

Acknowledgement

I would like to offer my deepest thanks to all the people who helped in the successful achievement of this project, QuickShow A Movie and Event Booking Platform. It has been a great learning experience and without the constant support, guidance, encouragement, and collaboration of many people, this project would not have happened. To begin with, I would like to express my utmost gratitude to my project guide and faculty mentor, Mrs. Swati Goel, who gave me the priceless advice, helpful feedback, and encouragement during the creation of this project. Their experience and recommendations were essential in developing the project to what it was. I would also wish to acknowledge the faculty members of the department, who provided me with the necessary resources, technical help, and academic support at various steps of this work. Their observations made me aware of the realities of web development, user interface development, database management, and documentation of projects. The institution/college administration should also be credited with sincerely providing me with an environment that fosters innovation, experimentation and practical project learning. The resources allowed me to test the latest web technologies and solutions needed to develop QuickShow. I owe a great debt to my peers and friends who continually encouraged me and gave valuable advice more so when we were in the testing and debugging stages. Their collaboration, discussions and feedbacks enabled them to enhance the application performance and usability. I would also like to give a highly special appreciation to my family whose unconditional support, patience, and encouragement enabled me to be focused and committed during the project journey. I have always been motivated by their trust in my abilities to achieve my accomplishments. Lastly, I would like to recognize all the resources of online documentation, research articles, open-source libraries, and community of development whose source of knowledge I had to rely on in order to learn the intricate concepts and apply multiple features of the QuickShow system. I would like to thank every individual who directly and indirectly took part in this project with all the sincerity. You have been a big part in the success and significance of QuickShow as a learning process.

Preface

The project report entitled as QuickShow Online Ticket Booking System is the product of thorough research, systematic analysis and practical development that was conducted as part of my academic program. The primary goal of the project is to acquire some hands-on experience in developing a modern, effective, and user-friendly web application that resembles the functionality of popular online ticket booking websites like BookMyShow. As digital technologies are rapidly developing, the manner in which individuals engage entertainment and events has changed considerably. The online booking of tickets has become a part of the daily routine because it offers convenience, easy access, and facilitates user experience. With this trend in mind, QuickShow was created as a one-stop shop where people can search movies, events and comedy shows, choose theatres, buy seats, transact safely and receive digital tickets- all in one system. In creating this project, I had to deal with web technologies such as HTML, CSS, JavaScript, Node.js, Express.js and MongoDB that are modern. Also, I learnt more by doing, integrating API features like secure payment systems, JWT authentication, and administrative features like content and user management. Functionalities such as the ability to choose a showtime, theatres, 2D/3D, interactive seating, and e-ticket downloading helped me to learn more about full-stack applications and user-friendly design. The report is clearly written, and the functional and technical parts of the system are explained in a well-structured way. It also seeks to provide the reader with a clear concept of how a real world web application is planned, designed and implemented with the use of modern tools and development practices. Every part of the report brings out the chronology of the project idea to implementation. This project has also further enhanced my technical expertise, and problem- solving skills, time management, and methodological approach to development. The entire experience has been very valuable and has provided a solid base in my future career in software development. I am wishing that this report will be of help to students, educators and those who might be interested in knowing about full-stack web development. It is my attempt to provide a practical, well documented and real world application model.

Contents

S.No	Topic
1	Acknowledgement
2	Preface
3	Introduction to Project
4	Feasibility Study
5	Requirement Analysis
6	Detailed Analysis of Modules
7	Design (LLD/Flowchart)
8	Web interface
9	Coding
10	Bibliography & Ref

Introduction to Project

Overview

QuickShow is a complete online ticketing system made to provide a hassle-free and easy-to-use experience in the style of popular ticketing systems such as BookMyShow. With the entertainment market moving more towards digital solutions, consumers are turning to platforms that enable them to easily search movies, schedules of shows, compare theatres, and reserve tickets immediately.

QuickShow is designed to live up to these expectations to offer an easy-to-use interface, secure transactions, and effective booking process. The system also includes major features like user authentication, whereby the user can sign up and log in to access his or her personal area. The users can navigate through the movies and events by language, genre, and format (2D/3D), as well as the detailed theatre listings and available time slots. One of the most significant features of QuickShow is the dynamic seat selection system, which demonstrates the availability of seats in real-time and allows selecting the desired seats with one click. The platform is also reliable as it has incorporated a secure payment system allowing the transfer of transactions to be made smoothly and safely. After making a booking, a digital ticket is sent to the user, which can be downloaded and presented on entry into the venue. Unlike unified movie booking sites, QuickShow provides the local comedy shows and events, which makes it a more effective entertainment hub.

The system offers a robust administrative dashboard enabling administrators to administer movies, events, theatres, seating arrangements, and booking history effectively on the administrative side. This will also keep the platform updated and organized at any given time. In general, QuickShow is designed to replicate a real-life booking system and its high focus on scalability, security, and user satisfaction.

Objective

The main goals of the QuickShow system are:

1. To design and develop an online entertainment booking system that will streamline the process of booking movies and events using simple and clean interface.
2. In order to use secure authentication mechanism, such as user registration, user login and session management, it is important to make sure that all actions are assigned to authenticated users.
3. To create an effective administration dashboard, where administrators can easily handle movies, events, schedules of shows, seating arrangements and other data on the platform without having the technical expertise to do so.
4. To offer an interactive seat booking service that enables the user to see and book available seats in real time, reducing mistakes and conflicts in booking.
5. To incorporate a secure and trusted system of payment that will facilitate seamless and encrypted transaction in making reservations of the tickets.
6. To provide a variety of viewing choices like 2D and 3D, allowing users the choice, depending on their preferences and availability.
7. To create instant digital tickets when a booking is successful, enabling the user to download and redeem it in the venue without any problem.
8. To serve and encourage local entertainment by incorporating a feature of browsing and purchasing of tickets to local events and comedy.
9. To come up with a scalable and responsive system that guarantees consistency in performance under different devices, such as desktops, tablets, and mobile phones.

Scope

The QuickShow project has several features and system functionalities, and they can be broadly divided into User, Admin, and Technical domain. The User Scope is concerned with the capabilities of the end users to interact with the platform.

◆User Scope

1. Registration and Authentication of users. The site enables people to form accounts, successfully log-in, reset passwords, and manage their accounts. This guarantees individual and secure user experience.
2. Investigating Films and Festival. Users can surf through the present and future movies, comedy shows and local events. The category of language, genre, format, ratings and user reviews can be used to filter content.
3. Cinema and Showtime Choice. The list of the available theatres, dates, and show timings are shown on each movie or event page whereby a user can select something that most fits their schedule.
4. Interactive Seat Booking The system offers a visual seat arrangement, in which the users can be able to see the availability of seats in real time. The seat types (available, booked, premium) are well marked and one can easily select the type of seat and continue to pay.
5. Digital Ticket Access Upon a successful booking, users will be sent an electronic ticket, which includes information like seat number and QR code. This ticket is downloadable or can be viewed at any time using the user dashboard.
6. Booking Management Through their account, users can track their booking history, do reviews of past reservations and manage future bookings.
7. Local Event Benefits. Besides films, users have the ability to find films and reserve tickets to local comedy shows and local events, which assists in marketing local entertainment.

◆Admin Scope

1. **Movie and Event Management.**
   The administrators are in complete control to add, update, delete and arrange movies, comedy shows and events in the platform.
2. **Cinema and Show Management.**
   The system allows the admins to control the theatre information, seating arrangements, the screen sizes (2D and 3D) and the movie schedules.
3. **Booking- and Ticket Tracking.**
   Admins are able to check the booking of users, the availability of seats and to analyze the trends in bookings to facilitate smooth running of the system.
4. **User Account Management**
   The site offers features that allow administrators to see the profiles of users, manage their accounts through approvals or limited access to user accounts and monitor the activities of the users.
5. **Records and Reports of payments.**
   Administrators will be able to access the details of transactions, records of payments and to create reports or analytics to help in making better decisions.
6. **Maintenance and Updates of the system.**
   Admin panel facilitates routine maintenance of the system such as bug fixes, content updates and ensuring the platform performs optimally.

◆Technical Scope

1. Database Management

The system has a secure and optimized database to store all the necessary data which includes user information, films, booking, payment, events and seats.

2. Backend Processing

The backend provides the basic functionality of the service, including the processing of user requests, bookings, payments and the administrative functionality.

3. Frontend Design

An effective and interactive interface is created to make sure that it is easily accessible on various gadgets such as desktops, tablets, and smartphones.

4. Security Implementation

The system has high security measures like encryption of passwords, input checks, authentication tokens, and secure payment systems to secure user information as well as transactions.

Feasibility Study

A feasibility study is implemented to determine based on available resources, time and constraints whether the proposed system, which is an online ticket booking system, QuickShow can be developed and successfully implemented. It assists in determining the feasibility and the viability of the project. This analysis will be broken down into four main sections, which include technical, economical, operational and schedule feasibility.

1. Technical Feasibility

Technical feasibility is concerned with the availability of the tools, technologies and expertise required to develop and operate the QuickShow system. As the platform also has user authentication and real-time seat booking, administrative controls, and secure payment processing, it is necessary to make sure that all technical requirements can be met.

1.1 **Availability of Technology**

The technologies required for developing QuickShow are modern, reliable, and widely supported within the development community.

- **Frontend:** HTML5, CSS3, JavaScript, and frameworks like React
- **Backend:** Node.js and Express.js
- **Database:** MongoDB, MySQL, or PostgreSQL
- **Payment Integration:** Razorpay API, Stripe, or PayPal
- **Authentication:** JWT (JSON Web Tokens) and bcrypt for password security

All of these technologies are either open-source or easily accessible and are commonly used in real-world applications.

1.2 Availability of Technical Skills

The skills required to build QuickShow are commonly covered in academic curricula and are accessible to most development teams. These include:

- Web development (frontend and backend)
- Database design and management
- API integration
- Knowledge about secure payment systems.

- Creation of administration dashboards

Standard learning materials and hands-on experience can be used to develop these competencies.

1.3 System Performance Requirements

The system is expected to meet certain performance standards, including:

- Real-time updates for seat availability
- Secure and efficient payment processing
- Fast page loading and responsiveness
- Ability to handle multiple users simultaneously

The chosen technology stack is capable of meeting these performance requirements effectively.

Conclusion (Technical Feasibility)

Based on the availability of modern technologies, accessible development tools, and achievable skill requirements, the QuickShow project is technically feasible. The system can be developed and deployed efficiently within the given constraints.

2. Economic Feasibility

Economic feasibility is about evaluating if the benefits of the QuickShow system outweigh the cost of developing, implementing and operating it.

2.1 Development Cost

QuickShow's development cost is low as most technologies involved are freely available and open source.

The primary expenses include:

- Developers' time and effort
- Essentials such as internet and power
- Optional use of commercial tools (UI frameworks, design assets, etc.)

2.2 Deployment Cost

The deployment cost varies according to the hosting provider. But there are low-cost and even free options:

- Hosting: Vercel or Render (free or affordable options)
- MongoDB Atlas (free plan)
- Domain Name: Around ₹500-₹1000 per year (optional)
- Scaling (optional): Approximately ₹400-₹800 per month

2.3 Operational Cost Operational costs include:

- Server and database maintenance
- Periodic costs for payment gateway APIs
- Software upgrades, bug fixes, optimisations

Many of these expenses can be minimized using free-tiers and resource management.

2.4 Expected Benefits

The QuickShow system has a number of benefits such as:

- Provides an easy-to-use ticket purchasing experience
- Offers valuable experience in developing real applications
- Provides a solid project for job applications
- Provides a platform to develop into a startup business
- Improves academic and technical skills

2.5 Cost–Benefit Analysis

Given the minimal costs and the high benefits of learning, usability and scalability, the system produces a good return on investment.

Conclusion (Economic Feasibility) The project is economically feasible, since the investment is low and the benefits (pedagogical and operational) are high.

3. Operational Feasibility

Operational feasibility assesses the system's effectiveness in practice, and its ease of use for users and administrators.

3.1 User Acceptance

QuickShow has ease-of-use and simplicity at its core.
Users can:

- Easily register and log in
- Easily search for movies and events
- Choose a seat from a seat map
- Complete secure payments
- Instantly view and download tickets

The user interface is designed in a way that people are used to (like BookMyShow), making it easy to use.

3.2 Admin Efficiency

The admin panel aims to simplify operations and minimise manual work. It allows administrators to:

- Manage movie and event listings
- Set show times and theatre information
- Manage bookings and availability
- Manage users and track transactions

It has an easy-to-use interface, requiring minimal training.

3.3 Operations and Maintenance

The system is modular in architecture, making it easy to maintain and update. Developers can effectively:

• Include movie and event listings
• Alter seating arrangements
• Improved payment integrations
• Enhance UI/UX features

This flexibility ensures the platform remains up-to-date and performs well over time.

3.4 Suitability for Real World Usage

In real-world scenarios QuickShow can work well. It supports the following:

• Handling many users at the same time
• Handling large datasets of movies and events
• Offering quick and dependable booking services
• Scaling to other cities and event categories

Conclusion (Operational Feasibility)

The system is operationally feasible as it is user friendly, efficient for administrator, easy to maintain and can work effectively in the real environment.

4. Schedule feasibility

Check the schedule for feasibility, whether the project can be completed on time meeting all functional requirements.

4.1 Time Requirements

There are several steps to build a full ticket booking system like:

- Design of the user interface
- API development (backend)
- Database creation and upkeep
- Implementing authentication processes
- Design the logic for seat selection
- Integration of payment systems
- Developing the admin dashboard

4.2 Project Schedule

The development process can be completed within a structured 10 week timeline:

- Week 1-2: Requirement Gathering & UI/UX Design
- Weeks 3-4: Frontend development
- Week 5-6: Database set up and backend development
- Week 7: Integration of seat selection and payment
- Week 8: Admin panel implementation
- Week 9: Testing & Debugging
- Week 10: Documentation and last improvements This time line is realistic and achievable for a student project.

4.3 Team Efficiency

A small team of 2-4 members can efficiently handle the project by dividing responsibilities as follows:

- Front end development
- Back end development
- Database Administration
- Administrator features

Collaboration tools such as GitHub can also increase productivity and coordination.

4.4 Managing the Risk

Some of the challenges that can happen during development are:

- Payment gateway integration issues
- Complexity of implementation of seat selection logic
- Troubleshooting authentication errors

Such risks can be mitigated through careful planning, testing and time management.

Conclusion (Feasibility of Schedule)

QuickShow is feasible in terms of schedule because the scope of the project is well matched with the planned time and the system can be completed successfully within the given duration.

Detailed Analysis of Individual Module

The QuickShow platform is made up of many interconnected modules working together to provide a smooth and efficient movie and event booking experience. Each module is built with a specific functionality, structured data flow and scalable architecture to ensure reliability, performance and user satisfaction.

A detailed overview of the key modules is provided below:

1. User Authentication Module (Signup & Login)

This module handles user onboarding and ensures secure access to the system.

Key Features

- User registration with proper input validation
- Login using email or mobile number
- Secure password encryption and storage
- Detection and handling of duplicate accounts
- Input sanitization and authentication middleware

Workflow

1. The user submits the registration form.
2. The backend validates the input and checks for existing accounts.
3. The password is encrypted using hashing techniques.
4. User credentials are securely stored in the database.
5. During login, credentials are verified and a JWT/session token is generated.

Purpose

To provide a secure and seamless authentication process for both users and administrators.

2. Admin Portal Module

This module provides administrators with full control over platform management and operations.

Key Features

- Add, update, and delete movies or event data
- Manage theatres, show timings, and seat configurations
- Upload media such as posters and banners
- Monitor user activity and booking records
- Access payment reports and revenue insights

Workflow

1. The administrator logs in through secure credentials.
2. The dashboard displays key system metrics and summaries.
3. Admin performs CRUD operations (Create, Read, Update, Delete).
4. Changes are instantly reflected in the system database and user interface.

Purpose

To enable efficient management of content, users, and system operations.

3. Movie/Event Listing Module

This module is responsible for displaying all available movies and events to users.

Key Features

- Dynamic retrieval of movies and event data from the backend
- Filtering options based on language, genre, region, format (2D/3D), and ratings
- Display of content in card format with posters, titles, and details
- Pagination to improve performance and user experience

Workflow

1. The frontend sends a request to fetch movie/event data.
2. The backend retrieves relevant information from the database.
3. Data is processed and returned to the frontend.
4. The UI displays the content with filtering and sorting options.

Purpose

To present users with organized, searchable, and visually appealing entertainment options.

Internal Workflow

1. The frontend sends a request to the server for movie or event data.
2. The backend fetches the required information from the database.
3. The user interface presents the listings with options for sorting and filtering.

Purpose

To display relevant entertainment content in an organized and interactive format for users.

4. Show Time & Cinema Selection Module

This module allows users to select their preferred theatre and show timing.

Key Features

- Display of available cinemas
- Multiple show timings for each movie/event
- Options to choose formats such as 2D or 3D
- Indicators for real-time seat availability

Workflow

1. The user selects a movie or event.
2. The system retrieves available theatres and show schedules.
3. The user chooses a suitable cinema and time slot.
4. The backend temporarily stores session details for the booking process.

Purpose

To help users conveniently select the most suitable venue and timing based on their preferences.

5. Seat Selection & Booking Module

This module plays a critical role in enabling interactive seat booking.

Key Features

- Visual representation of seating layout
- Color-coded indicators for seat availability
- Pricing variations based on seat categories
- Real-time seat locking to prevent duplicate bookings

Workflow

1. Selected seats are temporarily reserved when chosen by the user.
2. If payment is successful, the seats are confirmed and permanently booked.
3. If the transaction fails or the user exits, the reserved seats are released.

Purpose
To ensure accurate, transparent, and conflict-free seat booking.

6. Secure Payment Module
This module manages all payment-related processes with a focus on security anreliability.

Key Features
- Integration with trusted payment gateways
- Support for multiple payment methods (UPI, cards, digital wallets)
- Automatic verification of transactions
- Secure token-based transaction handling
- Instant success or failure notifications

Workflow
1. The payment request is forwarded to the selected payment gateway.
2. The gateway processes and verifies the transaction.
3. On successful payment, the booking is confirmed and the ticket is generated.
4. On failure, reserved seats are released automatically.

Purpose
To provide a safe, smooth, and reliable payment experience for users.

7. Ticket Generation & Download Module
This module generates a digital ticket after successful booking.

Key Features
- Automatic generation of e-tickets
- QR code or barcode for verification
- Downloadable PDF format
- Includes booking details such as seat number, timing, venue, and transaction ID

Workflow
1. Booking details are collected and processed by the backend.
2. A ticket template is dynamically populated with relevant data.
3. The system generates a PDF and makes it available for download.

Purpose
To provide users with a secure and verifiable proof of their booking.

8. Local Events & Comedy Shows Module

This module extends the platform to include local entertainment events.

Key Features

- Listings for stand-up comedy, theatre, and cultural events
- Dedicated sections for different event categories
- Flexible pricing and scheduling
- Integration with the existing booking system

Workflow

1. Administrators upload event details to the system.
2. The platform displays these events in separate categories.
3. Users can book tickets using the same process as movie bookings.

Purpose

To support local artists and expand the platform's entertainment offerings.

9. User Dashboard & Profile Module

This module enables users to manage their account and booking information.

Key Features

- View past and upcoming bookings
- Edit and manage personal profile details
- Re-download tickets
- Access transaction history

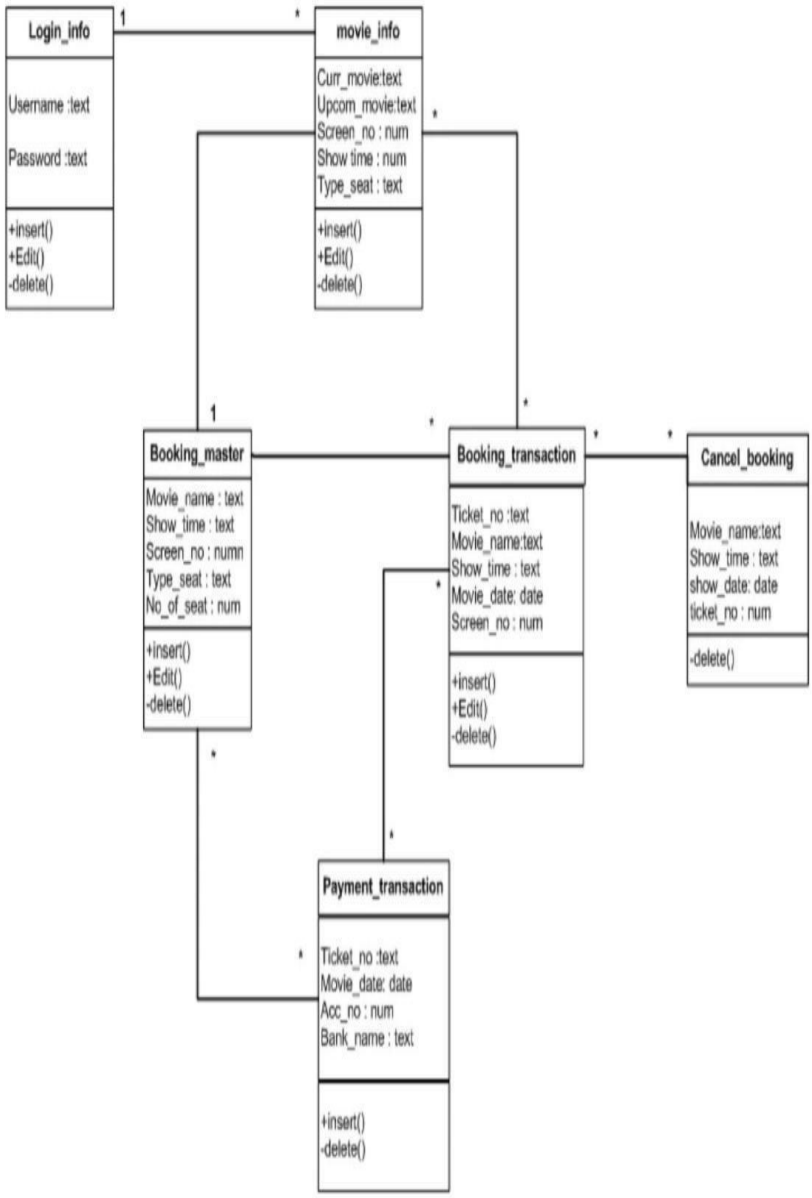
Workflow

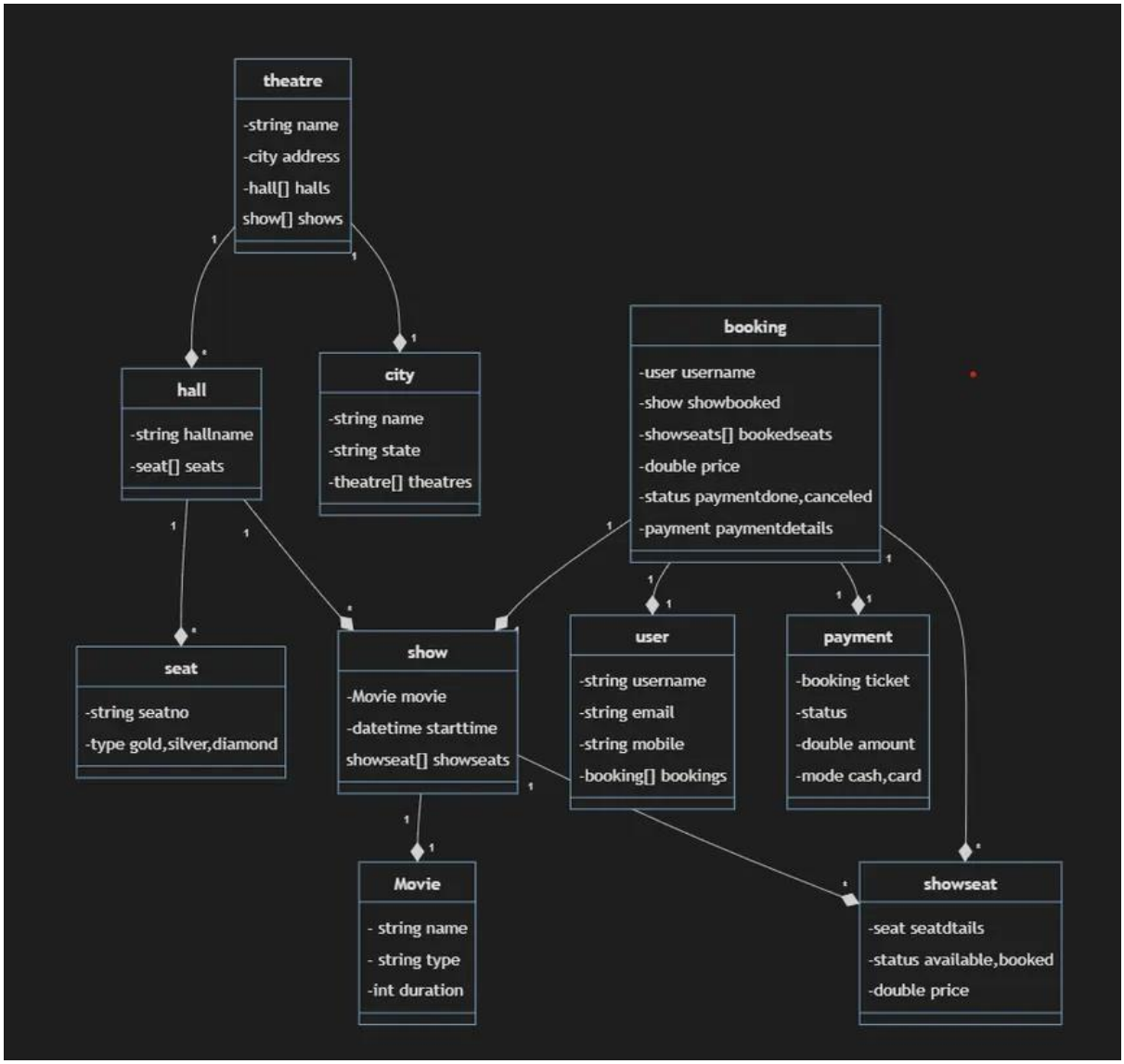
1. The user logs into their account and accesses the dashboard.
2. The system retrieves user data from the database.
3. Any updates made by the user are saved and reflected in real time.

Purpose

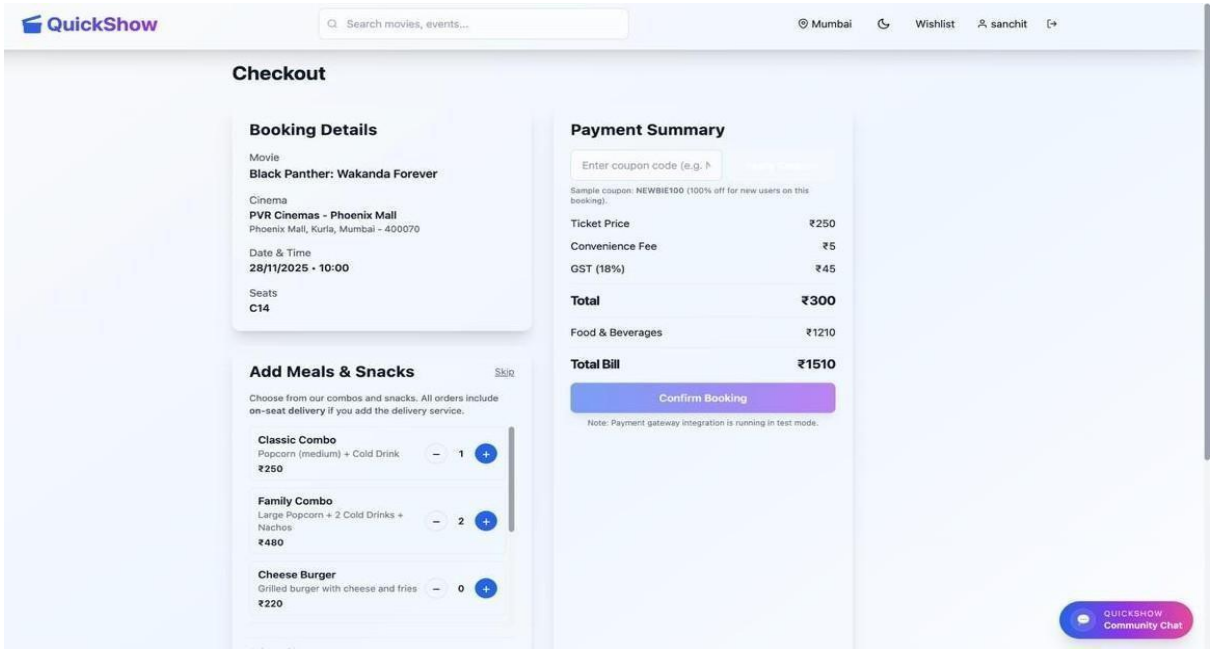
To enhance user experience by providing easy access to account and booking details.

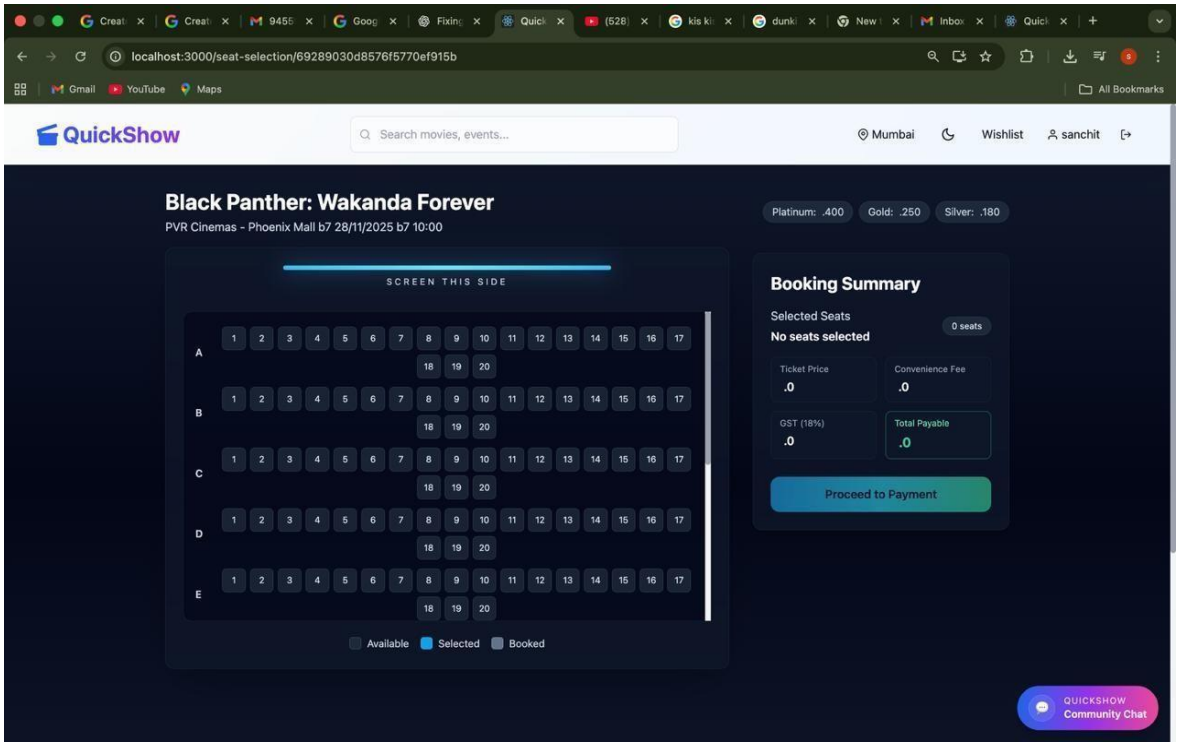
Class Diagram:

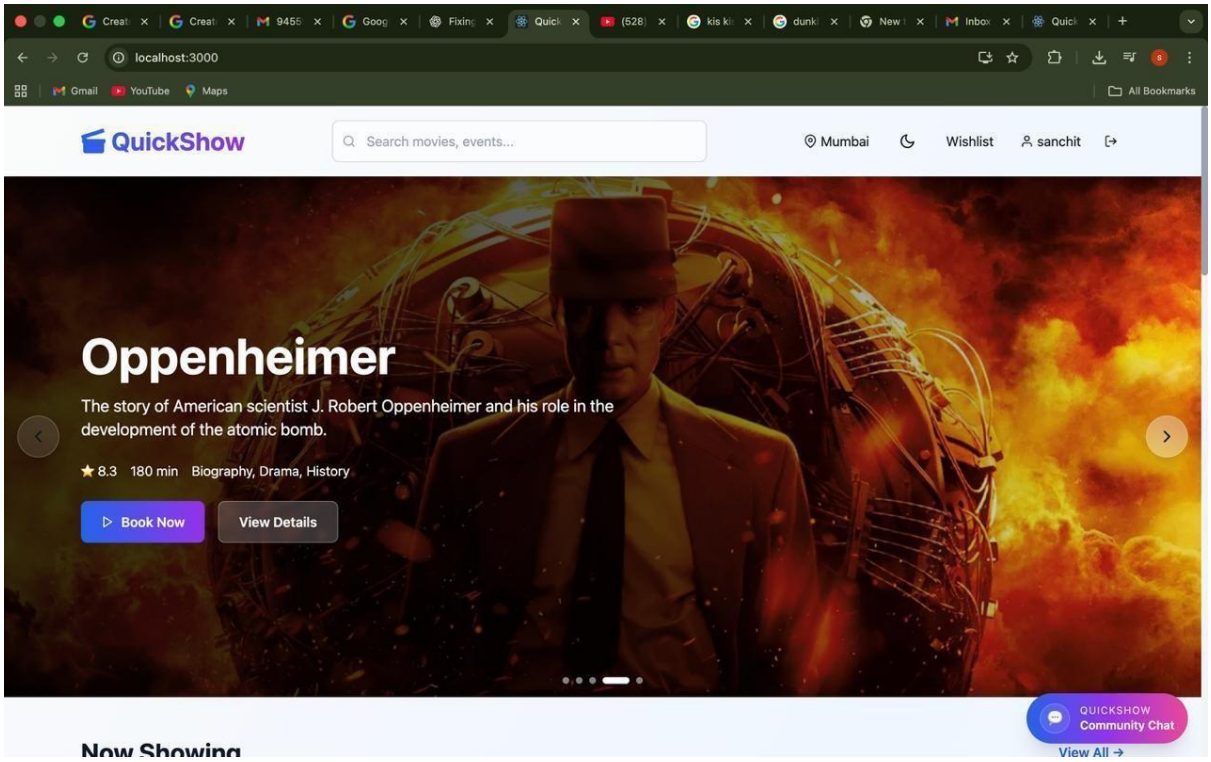


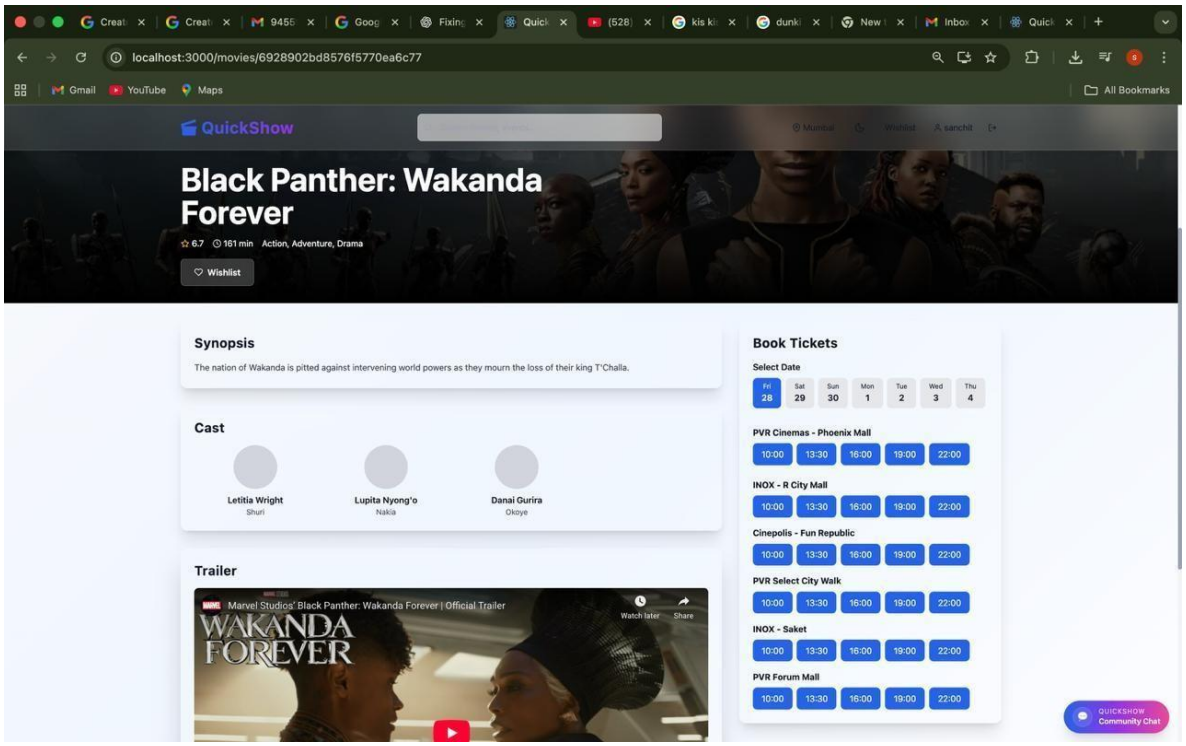

theatre
-string name
-city address
-hall[] halls
show[] shows
hall
-string hallname
-seat[] seats
city
-string name
-string state
-theatre[] theatres
booking
-user username
-show showbooked
-showseats[] bookedseats
-double price
-status paymentdone,canceled
-payment paymentdetails
seat
-string seatno
-type gold,silver,diamond
show
-Movie movie
-datetime starttime
showseat[] showseats
user
-string username
-string email
-string mobile
-booking[] bookings
payment
-booking ticket
-status
-double amount
-mode cash,card
Movie
- string name
- string type
-int duration
showseat
-seat seatdtails
-status available,booked
-double price

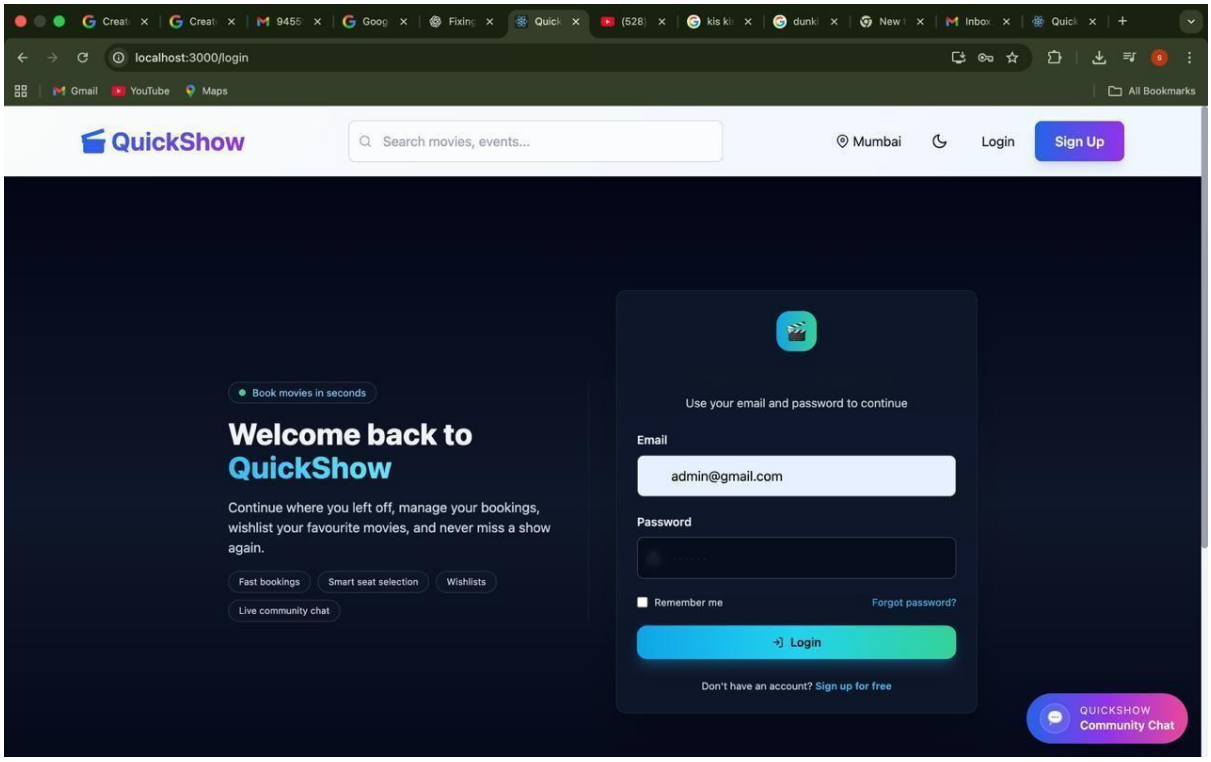
Web Interface

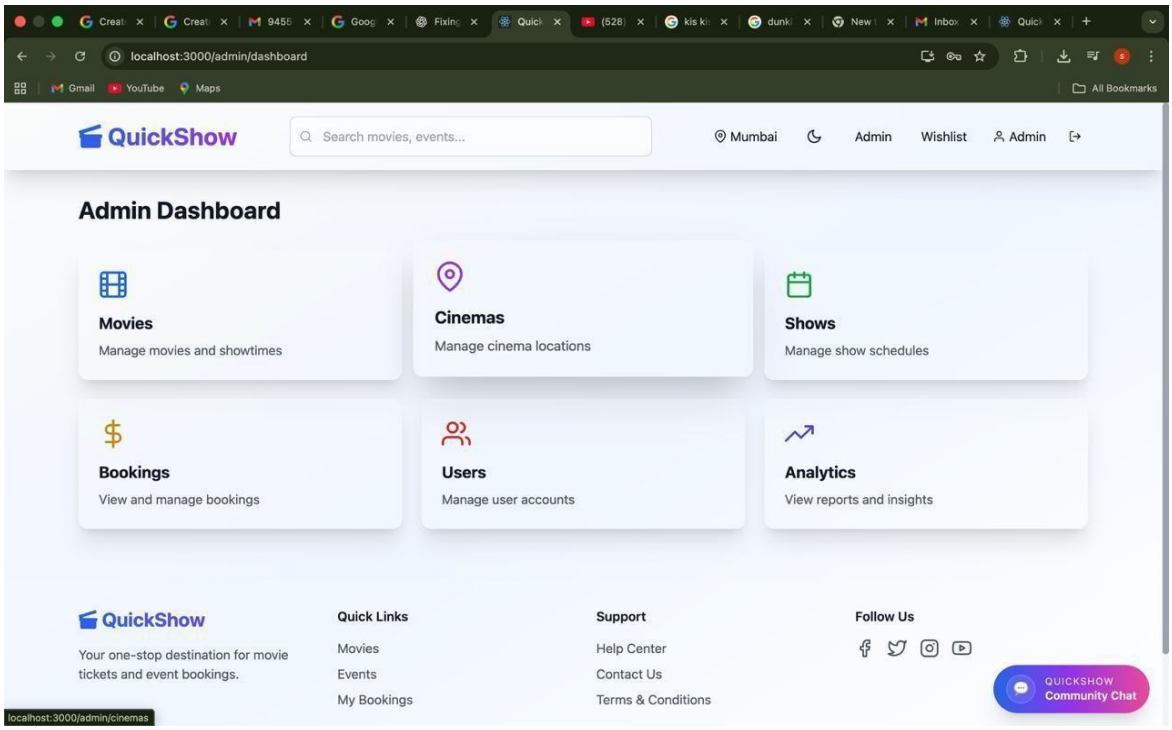


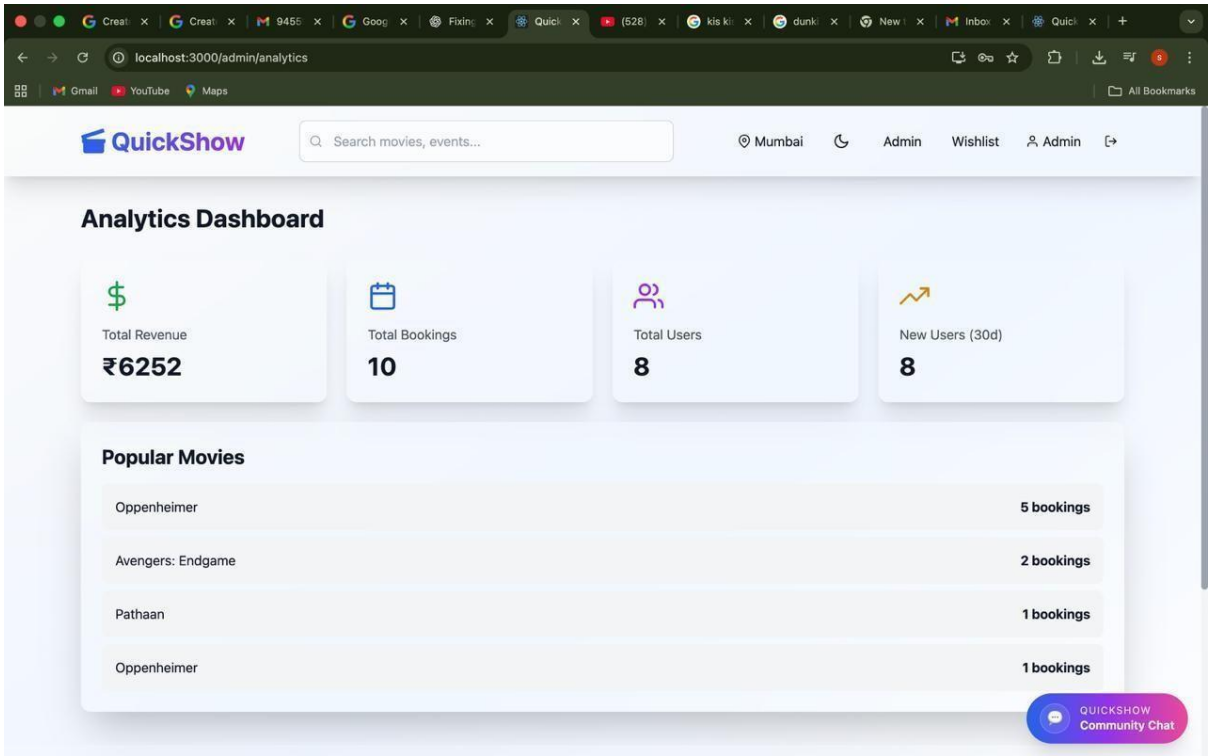


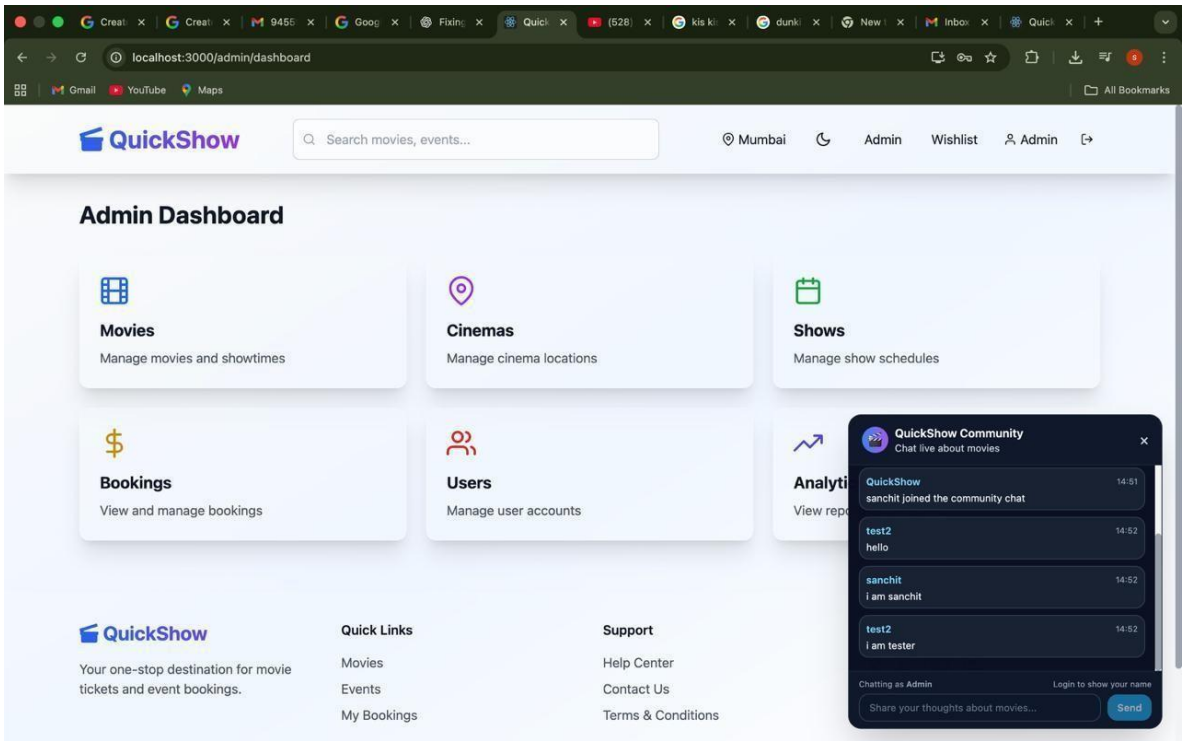

localhost:3000
QuickShow
Search movies, events...
Mumbai
Wishlist
sanchit
Oppenheimer
The story of American scientist J. Robert Oppenheimer and his role in the development of the atomic bomb.
8.3 180 min Biography, Drama, History
Book Now
View Details
QUICKSHOW Community Chat
Now Showing
View All


localhost:3000/movies/6928902bd8576f5770ea6c77
QuickShow
Black Panther: Wakanda Forever
6.7 161 min Action, Adventure, Drama
Wishlist
Synopsis
The nation of Wakanda is pitted against intervening world powers as they mourn the loss of their king T'Challa.
Cast
Letitia Wright
Shuri
Lupita Nyong'o
Nakia
Danai Gurira
Okoye
Trailer
Marvel Studios' Black Panther: Wakanda Forever | Official Trailer
WAKANDA FOREVER
Book Tickets
Select Date
Fri 28
Sat 29
Sun 30
Mon 1
Tue 2
Wed 3
Thu 4
PVR Cinemas - Phoenix Mall
INOX - R City Mall
Cinepolis - Fun Republic
PVR Select City Walk
INOX - Saket
PVR Forum Mall
10:00
13:30
16:00
19:00
22:00


localhost:3000/login
QuickShow
Search movies, events...
Mumbai
Login
Sign Up
Book movies in seconds
Welcome back to QuickShow
Continue where you left off, manage your bookings, wishlist your favourite movies, and never miss a show again.
Fast bookings
Smart seat selection
Wishlists
Live community chat
Use your email and password to continue
Email
admin@gmail.com
Password
Remember me
Forgot password?
Login
Don't have an account? Sign up for free
QUICKSHOW
Community Chat

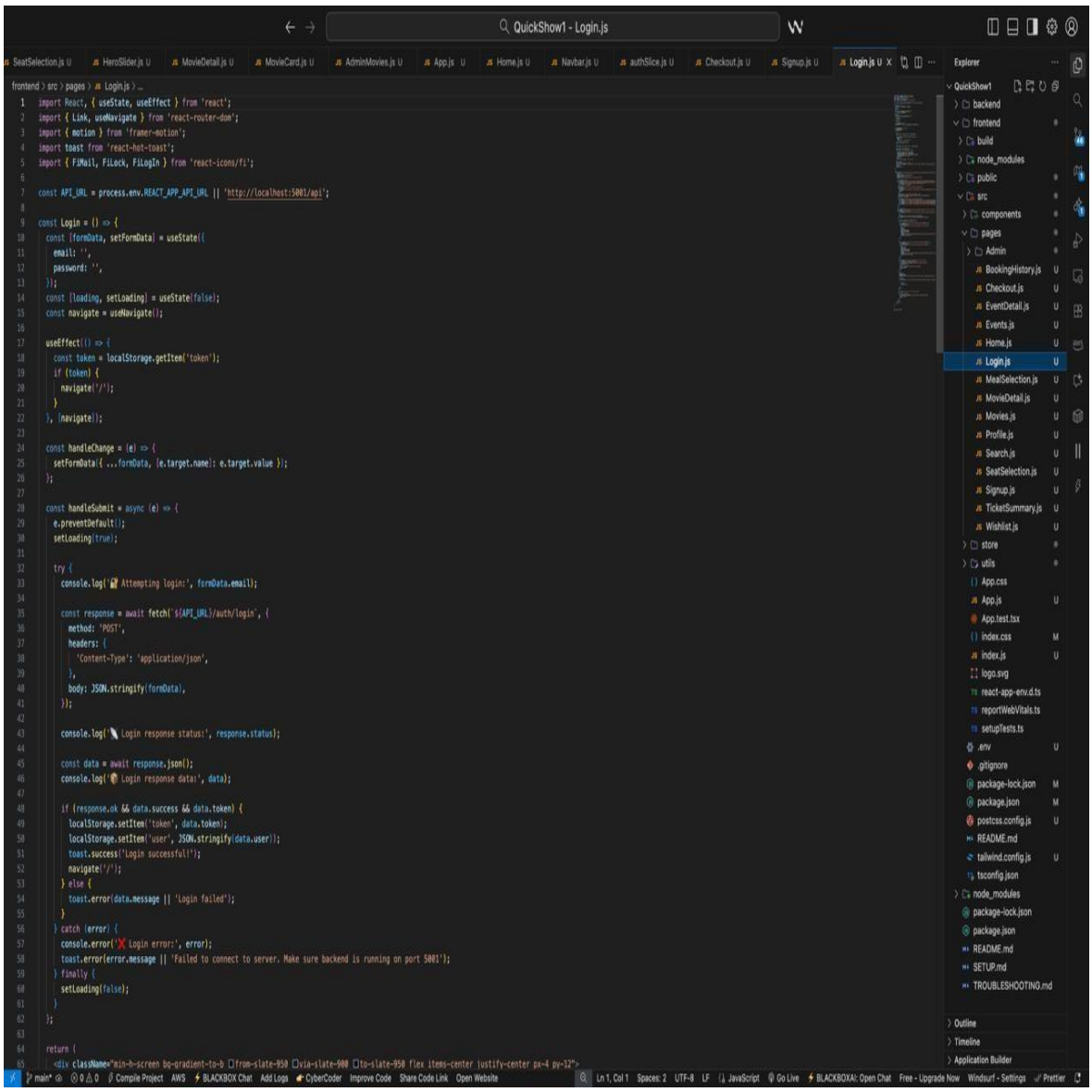

localhost:3000/admin/dashboard
QuickShow
Search movies, events...
Mumbai
Admin
Wishlist
Admin
Admin Dashboard
Movies
Manage movies and showtimes
Cinemas
Manage cinema locations
Shows
Manage show schedules
Bookings
View and manage bookings
Users
Manage user accounts
Analytics
View reports and insights
QuickShow
Your one-stop destination for movie tickets and event bookings.
Quick Links
Movies
Events
My Bookings
Support
Help Center
Contact Us
Terms & Conditions
Follow Us
QUICKSHOW
Community Chat
localhost:3000/admin/cinemas


localhost:3000/admin/analytics
QuickShow
Search movies, events...
Mumbai
Admin
Wishlist
Admin
Analytics Dashboard
Total Revenue
₹6252
Total Bookings
10
Total Users
8
New Users (30d)
8
Popular Movies
Oppenheimer
5 bookings
Avengers: Endgame
2 bookings
Pathaan
1 bookings
Oppenheimer
1 bookings
QUICKSHOW
Community Chat


localhost:3000/admin/dashboard
QuickShow
Search movies, events...
Mumbai
Admin
Wishlist
Admin
Admin Dashboard
Movies
Manage movies and showtimes
Cinemas
Manage cinema locations
Shows
Manage show schedules
Bookings
View and manage bookings
Users
Manage user accounts
QuickShow Community
Chat live about movies
QuickShow
sanchit joined the community chat
14:51
test2
hello
14:52
sanchit
i am sanchit
14:52
test2
i am tester
14:52
Chatting as Admin
Login to show your name
Share your thoughts about movies...
Send
QuickShow
Your one-stop destination for movie tickets and event bookings.
Quick Links
Movies
Events
My Bookings
Support
Help Center
Contact Us
Terms & Conditions

CODING

Frontend :



QuickShow1 - Checkout.js

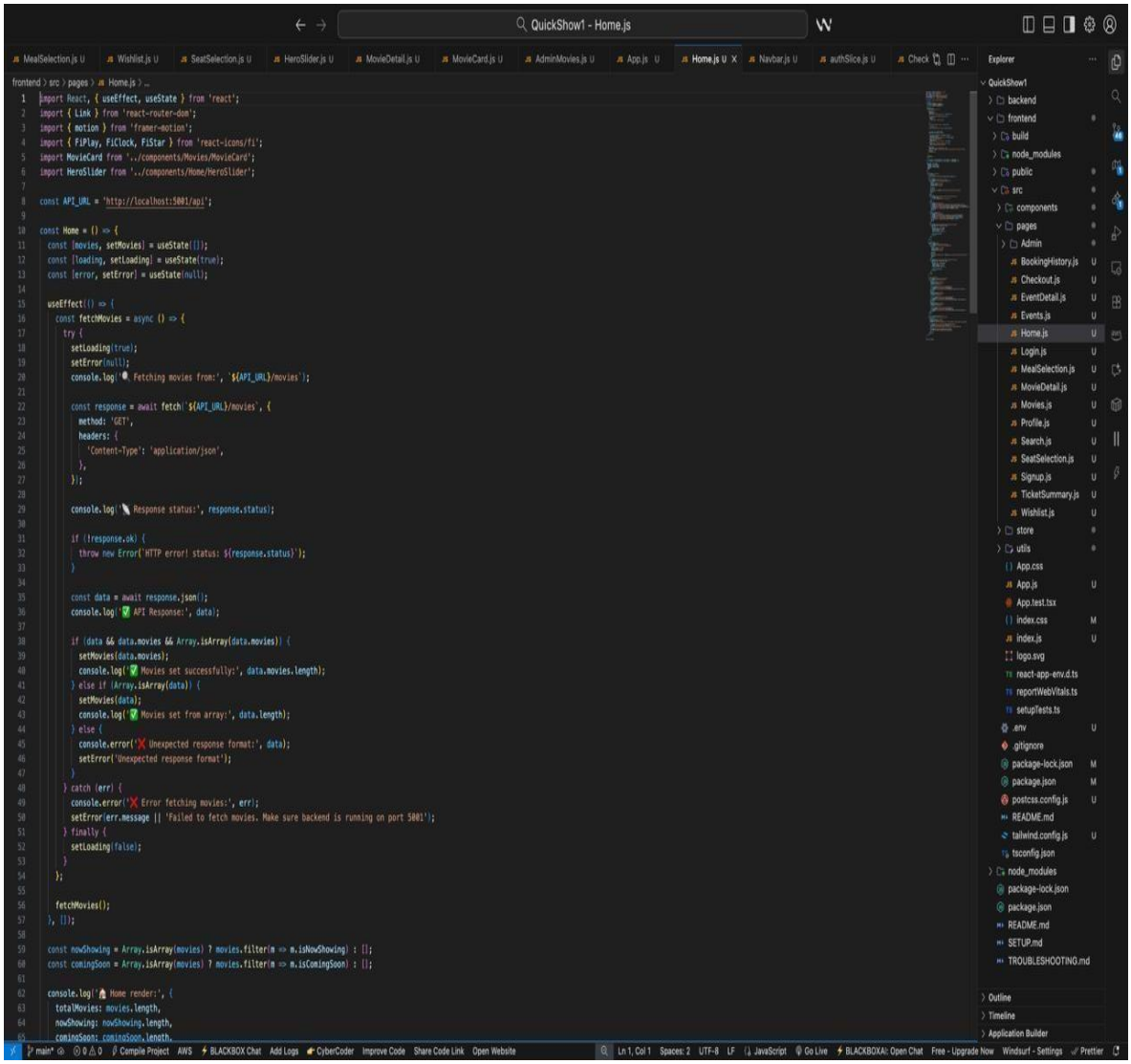
SeatSelection.js U | HeroSlider.js U | MovieDetail.js U | MovieCard.js U | AdminMovies.js U | App.js U | Home.js U | Navbar.js U | authSlice.js U | Checkout.js U | Signup.js U | Login.js U

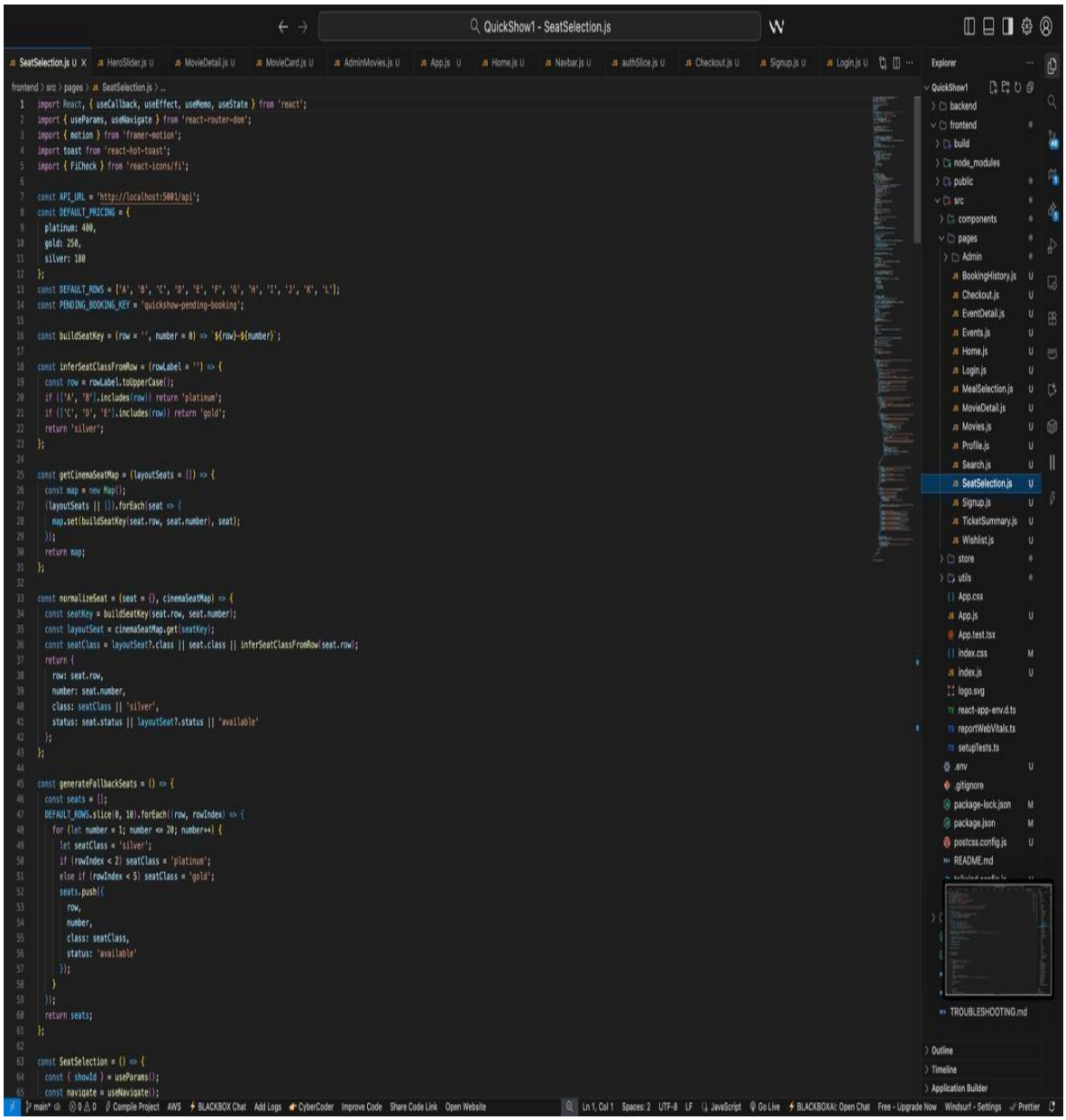
frontend > src > pages > Checkout.js > ...

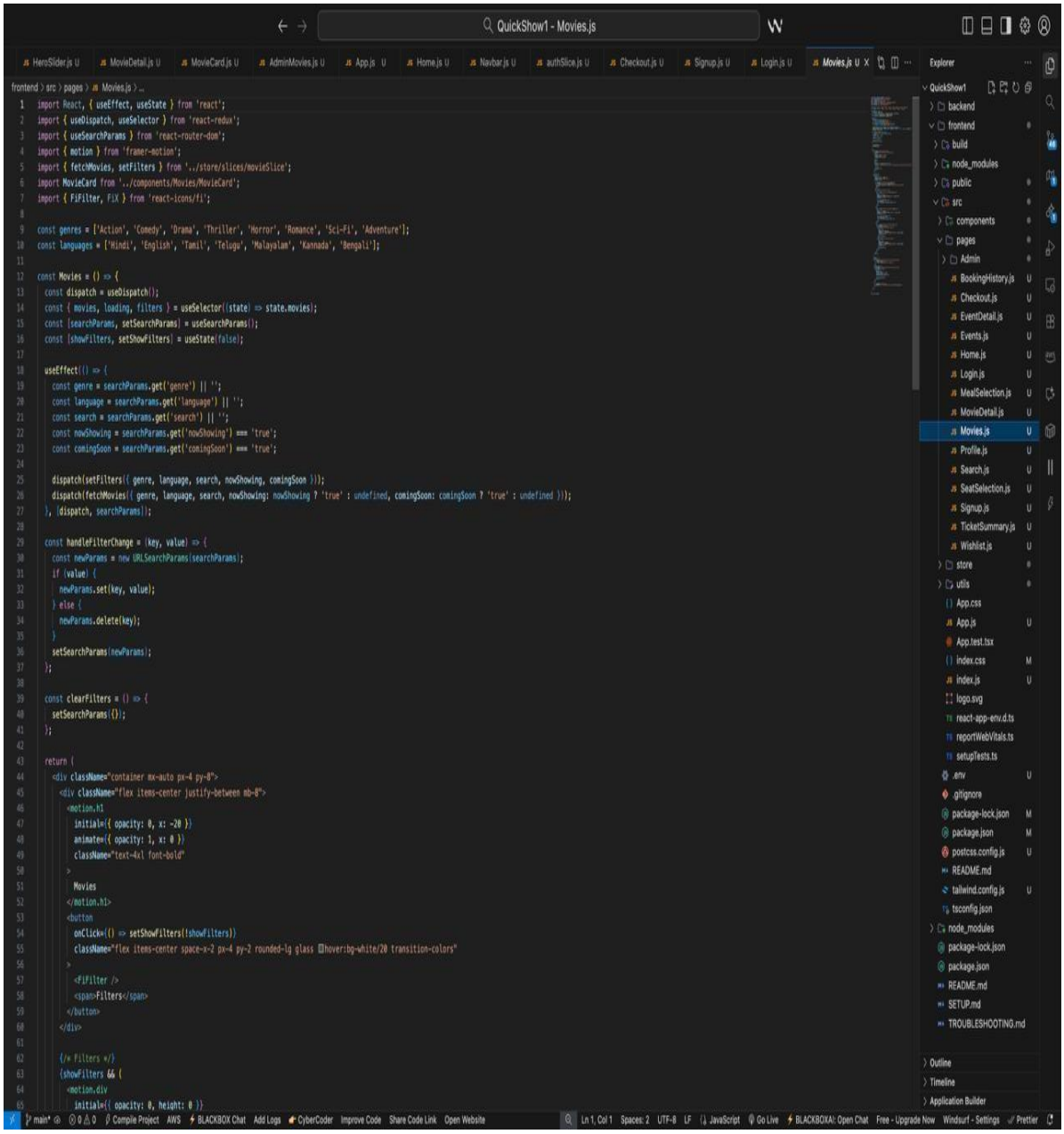
```
import React, { useEffect, useMemo, useState } from 'react';
import { useNavigate, useLocation } from 'react-router-dom';
import { motion } from 'framer-motion';
import toast from 'react-hot-toast';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5001/api';
const DEFAULT_PRICING = {
  platinum: 400,
  gold: 250,
  silver: 180
};
const PENDING_BOOKING_KEY = 'quickshow-pending-booking';

const MEAL_ITEMS = [
  {
    id: 'combo1',
    name: 'Classic Combo',
    description: 'Popcorn (medium) + Cold Drink',
    price: 250,
    type: 'combo',
  },
  {
    id: 'combo2',
    name: 'Family Combo',
    description: 'Large Popcorn + 2 Cold Drinks + Nachos',
    price: 480,
    type: 'combo',
  },
  {
    id: 'burger',
    name: 'Cheese Burger',
    description: 'Grilled burger with cheese and fries',
    price: 220,
    type: 'single',
  },
  {
    id: 'pizza',
    name: 'Margherita Pizza',
    description: '8" classic cheese pizza',
    price: 320,
    type: 'single',
  },
  {
    id: 'cold_drink',
    name: 'Cold Drink',
    description: '500ml chilled beverage',
    price: 90,
    type: 'single',
  },
  {
    id: 'on_seat_delivery',
    name: 'On-seat Delivery Service',
    description: 'Have your order delivered directly to your seat',
    price: 40,
    type: 'service',
  },
];

const AVAILABLE_COUPONS = [
  {
    code: 'NEWBIE100',
    type: 'percent',
    value: 100,
    description: '100% off for new users on first booking'
  }
```

Explorer
QuickShow1
backend
frontend
build
node_modules
public
src
components
pages
Admin
BookingHistory.js
Checkout.js
EventDetail.js
Events.js
Home.js
Login.js
MealSelection.js
MovieDetail.js
Movies.js
Profile.js
Search.js
SeatSelection.js
Signup.js
TicketSummary.js
Wishlist.js
store
utils
App.css
App.js
App.test.tsx
index.css
index.js
logo.svg
react-app-env.d.ts
reportWebVitals.ts
setupTests.ts
.env
.gitignore
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.json
node_modules
package-lock.json
package.json
README.md
SETUP.md
TROUBLESHOOTING.md
Outline
Timeline
Application Builder

main* | 0 0 | Compile Project | AWS | BLACKBOX Chat | Add Logs | CyberCoder | Improve Code | Share Code Link | Open Website | Ln 1, Col 1 | Spaces: 2 | UTF-8 | LF | JavaScript | Go Live | BLACKBOXAI: Open Chat | Free - Upgrade Now | Windsurf - Settings | Prettier

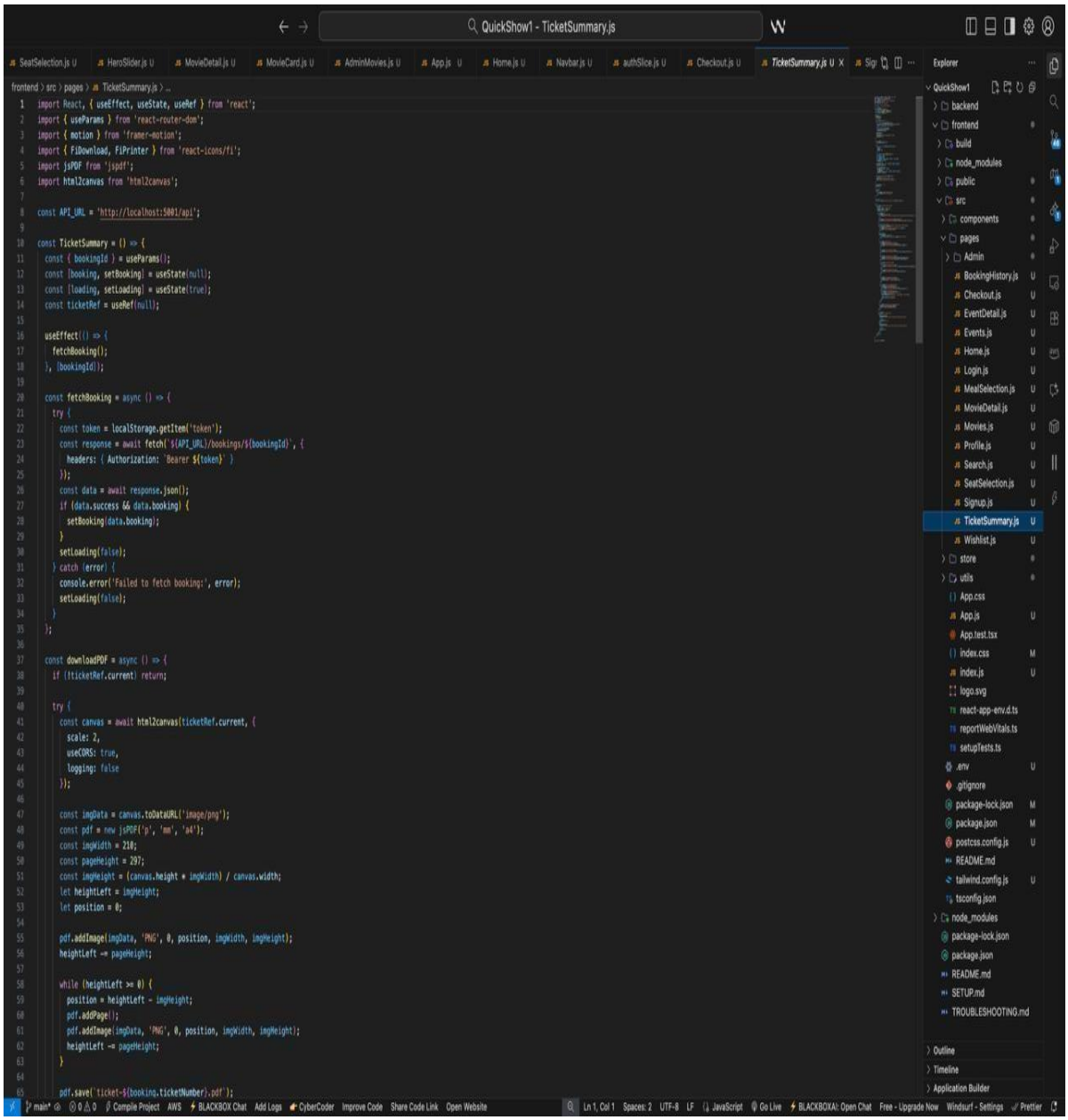

QuickShow1 - Home.js
MealSelection.js U
Wishlist.js U
SeatSelection.js U
HeroSlider.js U
MovieDetail.js U
MovieCard.js U
AdminMovies.js U
App.js U
Home.js U
Navbar.js U
authSlice.js U
Check
Explorer
QuickShow1
backend
frontend
build
node_modules
public
src
components
pages
Admin
BookingHistory.js
Checkout.js
EventDetail.js
Events.js
Home.js
Login.js
MealSelection.js
MovieDetail.js
Movies.js
Profile.js
Search.js
SeatSelection.js
Signup.js
TicketSummary.js
Wishlist.js
store
utils
App.css
App.js
App.test.tsx
index.css
index.js
logo.svg
react-app-env.d.ts
reportWebVitals.ts
setupTests.ts
.env
.gitignore
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.json
node_modules
package-lock.json
package.json
README.md
SETUP.md
TROUBLESHOOTING.md
Outline
Timeline
Application Builder
frontend > src > pages > Home.js > ...
import React, { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { motion } from 'framer-motion';
import { FiPlay, FiClock, FiStar } from 'react-icons/fi';
import MovieCard from '../components/Movies/MovieCard';
import HeroSlider from '../components/Home/HeroSlider';
const API_URL = 'http://localhost:5001/api';
const Home = () => {
const [movies, setMovies] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
useEffect(() => {
const fetchMovies = async () => {
try {
setLoading(true);
setError(null);
console.log('🔍 Fetching movies from:', `${API_URL}/movies`);
const response = await fetch(`${API_URL}/movies`, {
method: 'GET',
headers: {
'Content-Type': 'application/json',
},
});
console.log('📡 Response status:', response.status);
if (!response.ok) {
throw new Error(`HTTP error! status: ${response.status}`);
}
const data = await response.json();
console.log('✅ API Response:', data);
if (data && data.movies && Array.isArray(data.movies)) {
setMovies(data.movies);
console.log('✅ Movies set successfully:', data.movies.length);
} else if (Array.isArray(data)) {
setMovies(data);
console.log('✅ Movies set from array:', data.length);
} else {
console.error('❌ Unexpected response format:', data);
setError('Unexpected response format');
}
} catch (err) {
console.error('❌ Error fetching movies:', err);
setError(err.message || 'Failed to fetch movies. Make sure backend is running on port 5001');
} finally {
setLoading(false);
}
};
fetchMovies();
}, []);
const nowShowing = Array.isArray(movies) ? movies.filter(m => m.isNowShowing) : [];
const comingSoon = Array.isArray(movies) ? movies.filter(m => m.isComingSoon) : [];
console.log('🏠 Home render:', {
totalMovies: movies.length,
nowShowing: nowShowing.length,
comingSoon: comingSoon.length,
Ln 1, Col 1
Spaces: 2
UTF-8
LF
JavaScript
Go Live
BLACKBOXAI: Open Chat
Free - Upgrade Now
Windsurf - Settings
Prettier


QuickShow1 - SeatSelection.js
SeatSelection.js U
HeroSlider.js U
MovieDetail.js U
MovieCard.js U
AdminMovies.js U
App.js U
Home.js U
Navbar.js U
authSlice.js U
Checkout.js U
Signup.js U
Login.js U
frontend > src > pages > SeatSelection.js > ...
import React, { useCallback, useEffect, useMemo, useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { motion } from 'framer-motion';
import toast from 'react-hot-toast';
import { FiCheck } from 'react-icons/fi';
const API_URL = 'http://localhost:5001/api';
const DEFAULT_PRICING = {
platinum: 400,
gold: 250,
silver: 180
};
const DEFAULT_ROWS = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L'];
const PENDING_BOOKING_KEY = 'quickshow-pending-booking';
const buildSeatKey = (row = '', number = 0) => `${row}-${number}`;
const inferSeatClassFromRow = (rowLabel = '') => {
const row = rowLabel.toUpperCase();
if (['A', 'B'].includes(row)) return 'platinum';
if (['C', 'D', 'E'].includes(row)) return 'gold';
return 'silver';
};
const getCinemaSeatMap = (layoutSeats = []) => {
const map = new Map();
(layoutSeats || []).forEach(seat => {
map.set(buildSeatKey(seat.row, seat.number), seat);
});
return map;
};
const normalizeSeat = (seat = {}, cinemaSeatMap) => {
const seatKey = buildSeatKey(seat.row, seat.number);
const layoutSeat = cinemaSeatMap.get(seatKey);
const seatClass = layoutSeat?.class || seat.class || inferSeatClassFromRow(seat.row);
return {
row: seat.row,
number: seat.number,
class: seatClass || 'silver',
status: seat.status || layoutSeat?.status || 'available'
};
};
const generateFallbackSeats = () => {
const seats = [];
DEFAULT_ROWS.slice(0, 10).forEach((row, rowIndex) => {
for (let number = 1; number <= 20; number++) {
let seatClass = 'silver';
if (rowIndex < 2) seatClass = 'platinum';
else if (rowIndex < 5) seatClass = 'gold';
seats.push({
row,
number,
class: seatClass,
status: 'available'
});
}
});
return seats;
};
const SeatSelection = () => {
const { showId } = useParams();
const navigate = useNavigate();
Explorer
QuickShow1
backend
frontend
build
node_modules
public
src
components
pages
Admin
BookingHistory.js
Checkout.js
EventDetail.js
Events.js
Home.js
Login.js
MealSelection.js
MovieDetail.js
Movies.js
Profile.js
Search.js
SeatSelection.js
Signup.js
TicketSummary.js
Wishlist.js
store
utils
App.css
App.js
App.test.tsx
index.css
index.js
logo.svg
react-app-env.d.ts
reportWebVitals.ts
setupTests.ts
.env
.gitignore
package-lock.json
package.json
postcss.config.js
README.md
TROUBLESHOOTING.md
Outline
Timeline
Application Builder

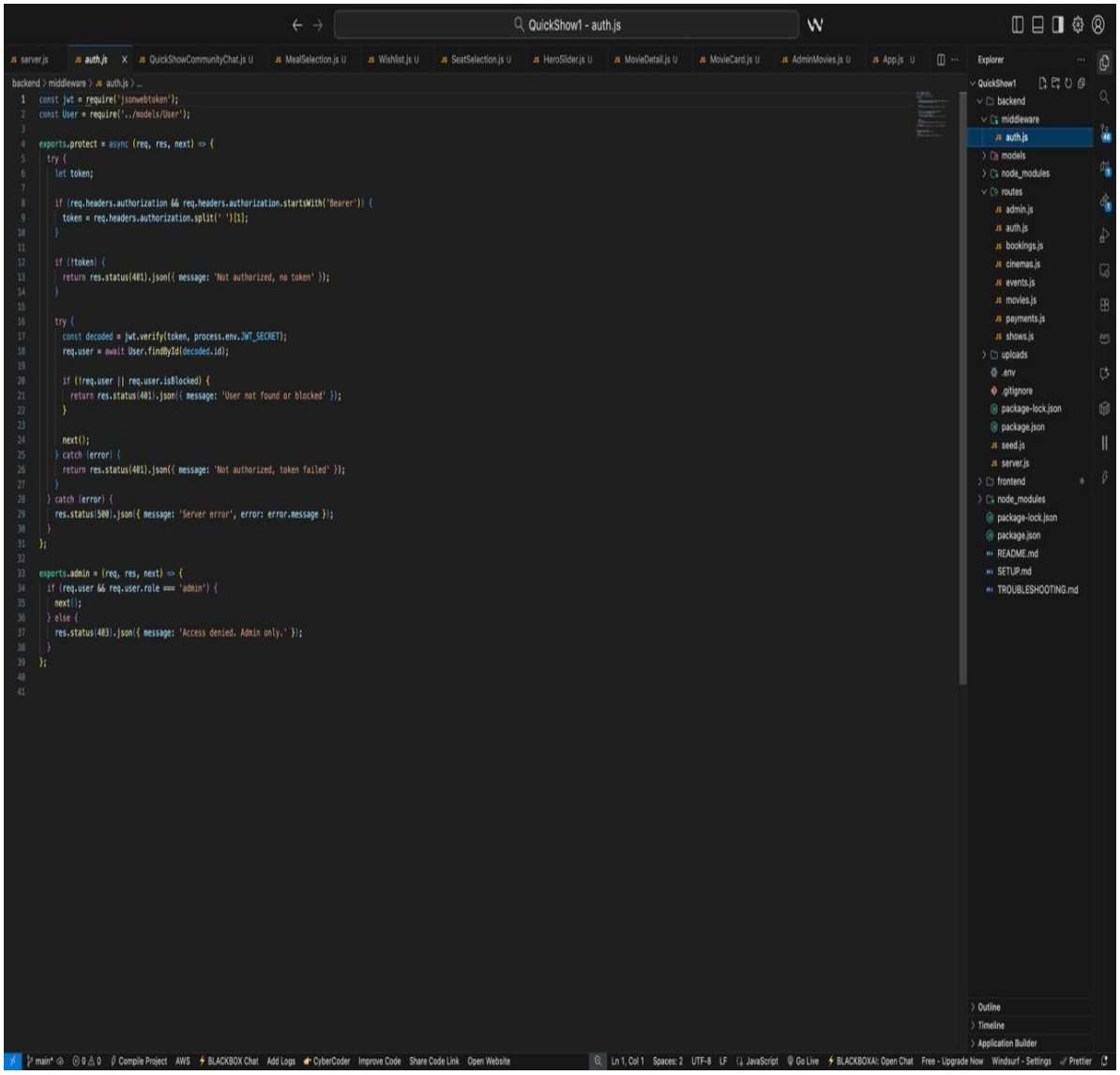

QuickShow1 - Movies.js
HeroSlider.js U
MovieDetail.js U
MovieCard.js U
AdminMovies.js U
App.js U
Home.js U
Navbar.js U
authSlice.js U
Checkout.js U
Signup.js U
Login.js U
Movies.js U
frontend > src > pages > Movies.js > ...
import React, { useEffect, useState } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { useSearchParams } from 'react-router-dom';
import { motion } from 'framer-motion';
import { fetchMovies, setFilters } from '../store/slices/movieSlice';
import MovieCard from '../components/Movies/MovieCard';
import { FiFilter, FiX } from 'react-icons/fi';
const genres = ['Action', 'Comedy', 'Drama', 'Thriller', 'Horror', 'Romance', 'Sci-Fi', 'Adventure'];
const languages = ['Hindi', 'English', 'Tamil', 'Telugu', 'Malayalam', 'Kannada', 'Bengali'];
const Movies = () => {
const dispatch = useDispatch();
const { movies, loading, filters } = useSelector(state => state.movies);
const [searchParams, setSearchParams] = useSearchParams();
const [showFilters, setShowFilters] = useState(false);
useEffect(() => {
const genre = searchParams.get('genre') || '';
const language = searchParams.get('language') || '';
const search = searchParams.get('search') || '';
const nowShowing = searchParams.get('nowShowing') === 'true';
const comingSoon = searchParams.get('comingSoon') === 'true';
dispatch(setFilters({ genre, language, search, nowShowing, comingSoon }));
dispatch(fetchMovies({ genre, language, search, nowShowing: nowShowing ? 'true' : undefined, comingSoon: comingSoon ? 'true' : undefined }));
}, [dispatch, searchParams]);
const handleFilterChange = (key, value) => {
const newParams = new URLSearchParams(searchParams);
if (value) {
newParams.set(key, value);
} else {
newParams.delete(key);
}
setSearchParams(newParams);
};
const clearFilters = () => {
setSearchParams({});
};
return (

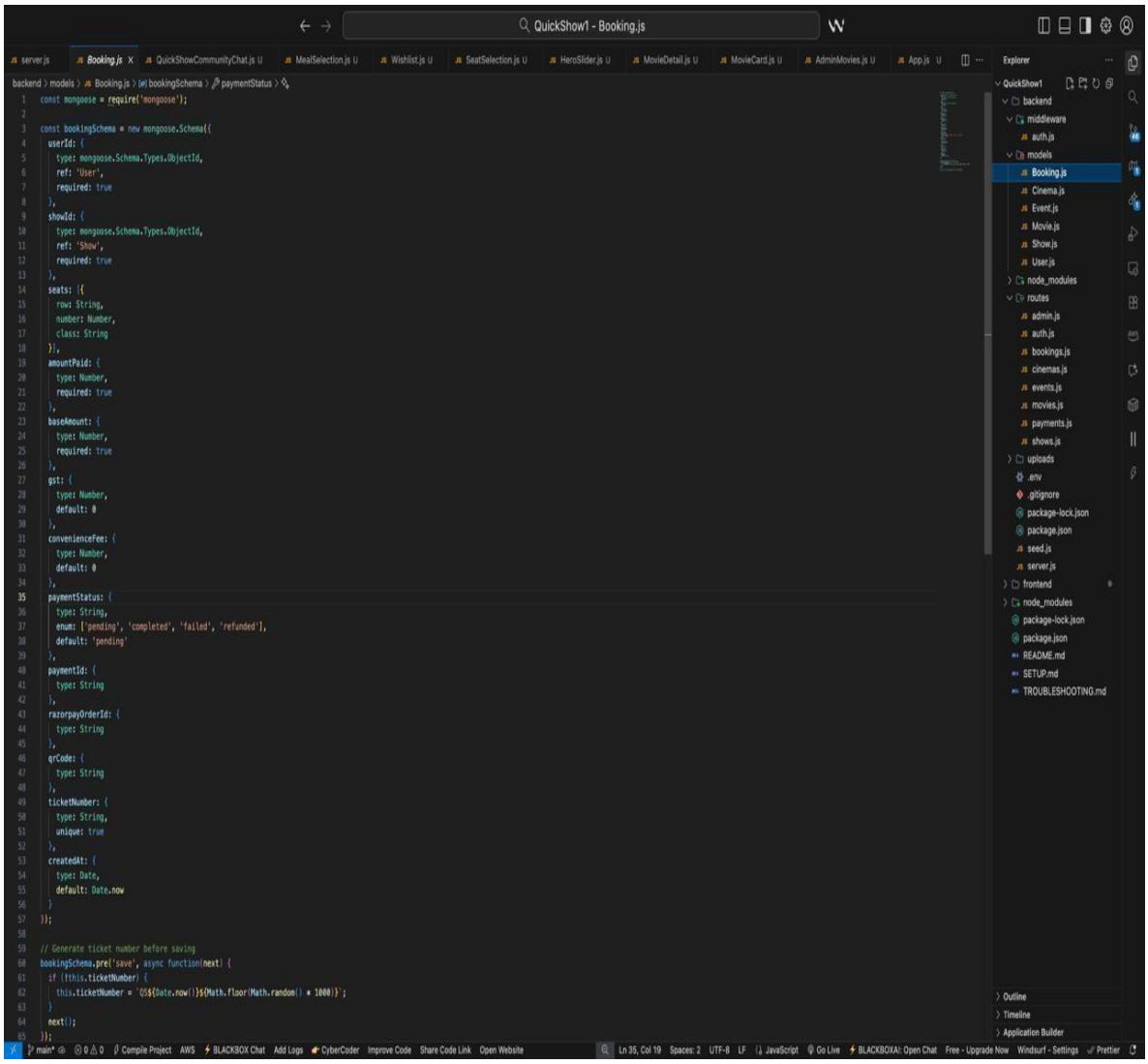
<motion.h1
initial={{ opacity: 0, x: -20 }}
animate={{ opacity: 1, x: 0 }}
className="text-4xl font-bold"
>
Movies
</motion.h1>
<button
onClick={() => setShowFilters(!showFilters)}
className="flex items-center space-x-2 px-4 py-2 rounded-lg glass hover:bg-white/20 transition-colors"
>
<FiFilter />
Filters
</button>

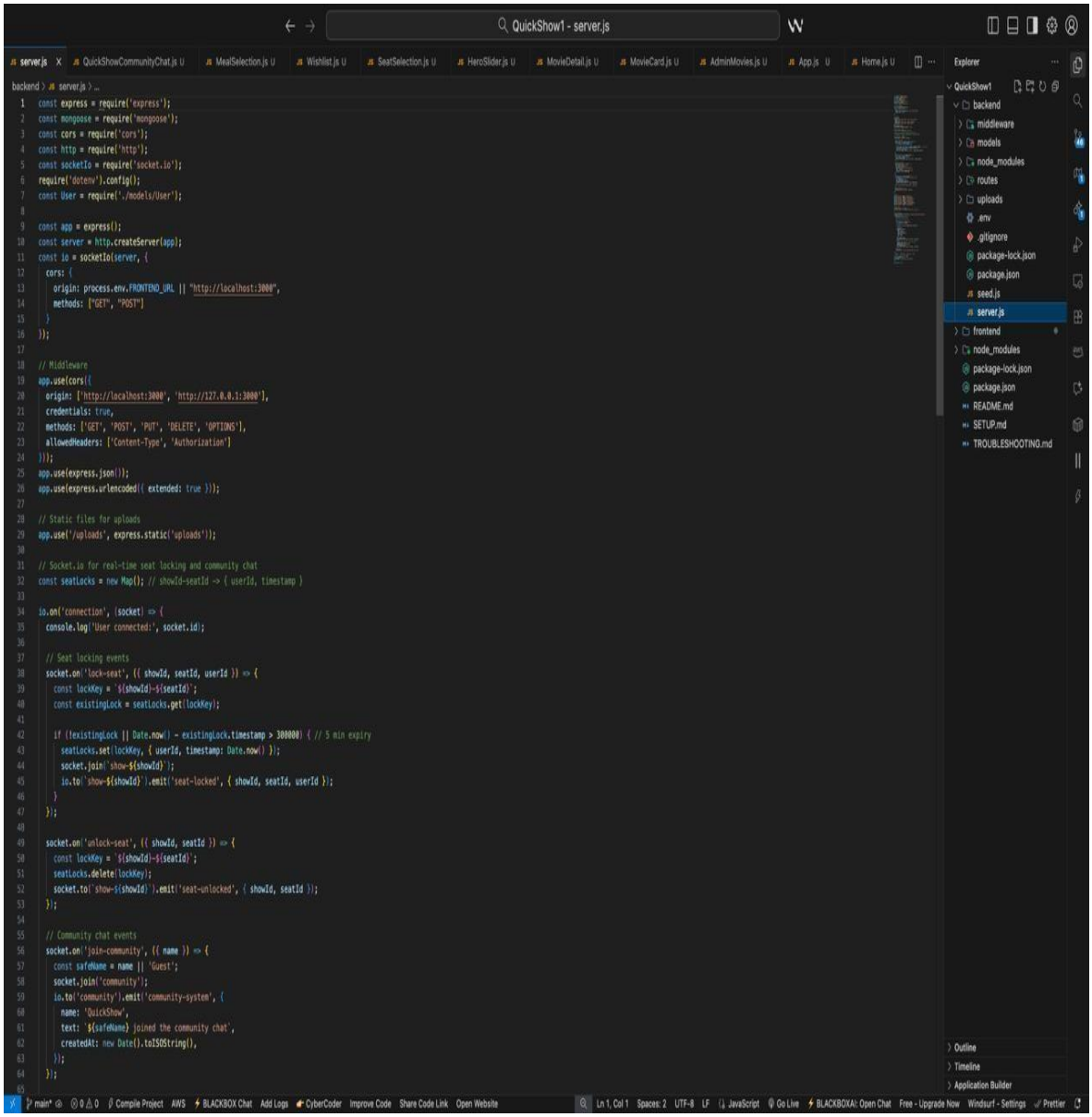
{/* Filters */}
{showFilters && (
<motion.div
initial={{ opacity: 0, height: 0 }}
Explorer
QuickShow1
backend
frontend
build
node_modules
public
src
components
pages
Admin
BookingHistory.js
Checkout.js
EventDetail.js
Events.js
Home.js
Login.js
MealSelection.js
MovieDetail.js
Movies.js
Profile.js
Search.js
SeatSelection.js
Signup.js
TicketSummary.js
Wishlist.js
store
utils
App.css
App.js
App.test.tsx
index.css
index.js
logo.svg
react-app-env.d.ts
reportWebVitals.ts
setupTests.ts
.env
.gitignore
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.json
node_modules
package-lock.json
package.json
README.md
SETUP.md
TROUBLESHOOTING.md
Outline
Timeline
Application Builder

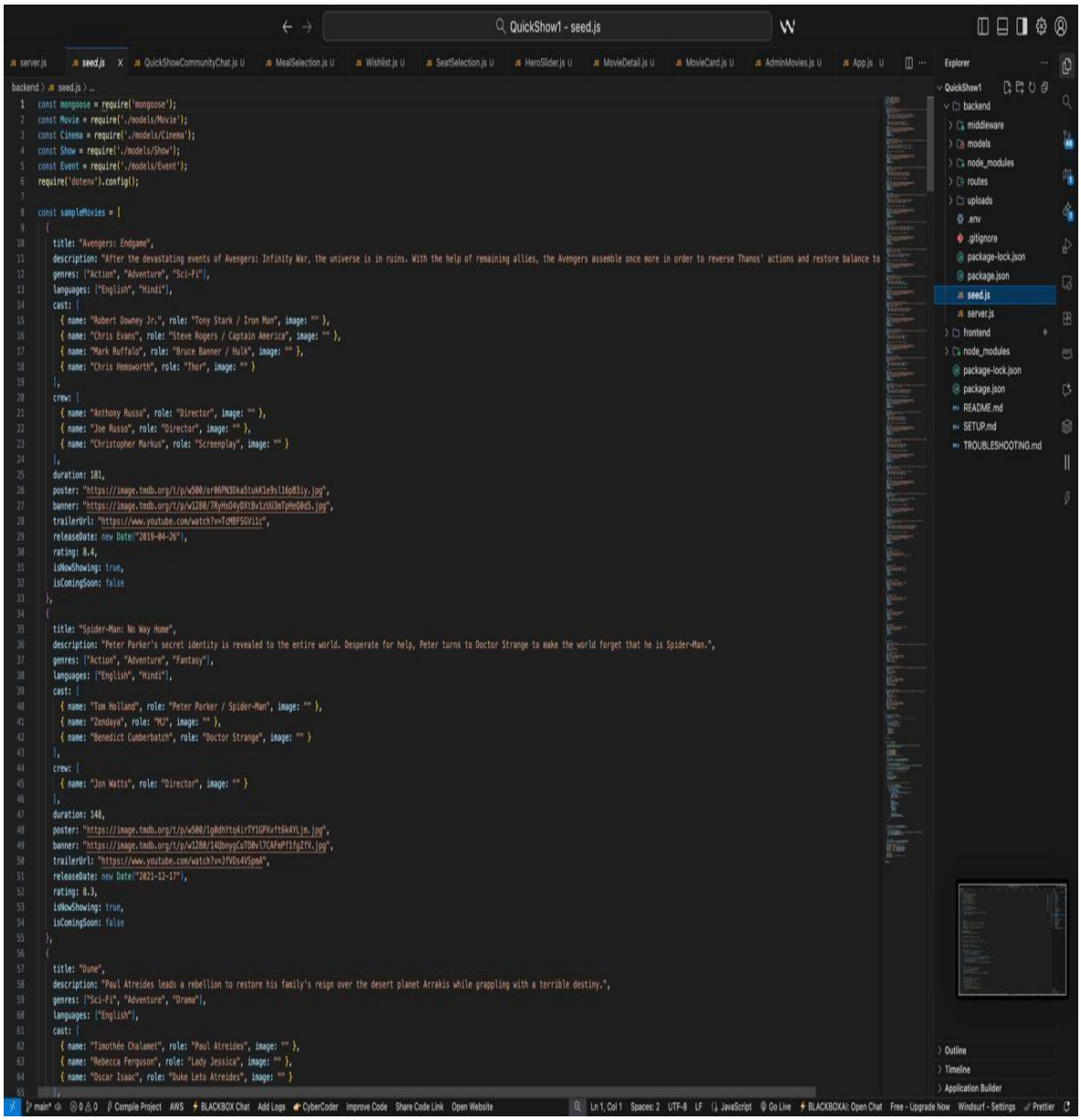

QuickShow1 - TicketSummary.js
SeatSelection.js U
HeroSlider.js U
MovieDetail.js U
MovieCard.js U
AdminMovies.js U
App.js U
Home.js U
Navbar.js U
authSlice.js U
Checkout.js U
TicketSummary.js U
Explorer
QuickShow1
backend
frontend
build
node_modules
public
src
components
pages
Admin
BookingHistory.js
Checkout.js
EventDetail.js
Events.js
Home.js
Login.js
MealSelection.js
MovieDetail.js
Movies.js
Profile.js
Search.js
SeatSelection.js
Signup.js
TicketSummary.js
Wishlist.js
store
utils
App.css
App.js
App.test.tsx
index.css
index.js
logo.svg
react-app-env.d.ts
reportWebVitals.ts
setupTests.ts
.env
.gitignore
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
tsconfig.json
node_modules
package-lock.json
package.json
README.md
SETUP.md
TROUBLESHOOTING.md
Outline
Timeline
Application Builder

BACKEND


QuickShow1 - auth.js
server.js
auth.js
QuickShowCommunityChat.js U
MealSelection.js U
Wishlist.js U
SeatSelection.js U
HeroSlider.js U
MovieDetail.js U
MovieCard.js U
AdminMovies.js U
App.js U
backend > middleware > auth.js > ...
const jwt = require('jsonwebtoken');
const User = require('../models/User');
exports.protect = async (req, res, next) => {
try {
let token;
if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
token = req.headers.authorization.split(' ')[1];
}
if (!token) {
return res.status(401).json({ message: 'Not authorized, no token' });
}
try {
const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = await User.findById(decoded.id);
if (!req.user || req.user.isBlocked) {
return res.status(401).json({ message: 'User not found or blocked' });
}
next();
} catch (error) {
return res.status(401).json({ message: 'Not authorized, token failed' });
}
} catch (error) {
res.status(500).json({ message: 'Server error', error: error.message });
}
};
exports.admin = (req, res, next) => {
if (req.user && req.user.role === 'admin') {
next();
} else {
res.status(403).json({ message: 'Access denied. Admin only.' });
}
};
Explorer
QuickShow1
backend
middleware
auth.js
models
node_modules
routes
admin.js
auth.js
bookings.js
cinemas.js
events.js
movies.js
payments.js
shows.js
uploads
.env
.gitignore
package-lock.json
package.json
seed.js
server.js
frontend
node_modules
package-lock.json
package.json
README.md
SETUP.md
TROUBLESHOOTING.md
Outline
Timeline
Application Builder


QuickShow1 - Booking.js
server.js
Booking.js
QuickShowCommunityChat.js U
MealSelection.js U
Wishlist.js U
SeatSelection.js U
HeroSlider.js U
MovieDetail.js U
MovieCard.js U
AdminMovies.js U
App.js U
backend > models > Booking.js > bookingSchema > paymentStatus
const mongoose = require('mongoose');
const bookingSchema = new mongoose.Schema({
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: true
},
showId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Show',
required: true
},
seats: [{
row: String,
number: Number,
class: String
}],
amountPaid: {
type: Number,
required: true
},
baseAmount: {
type: Number,
required: true
},
gst: {
type: Number,
default: 0
},
convenienceFee: {
type: Number,
default: 0
},
paymentStatus: {
type: String,
enum: ['pending', 'completed', 'failed', 'refunded'],
default: 'pending'
},
paymentId: {
type: String
},
razorpayOrderId: {
type: String
},
qrCode: {
type: String
},
ticketNumber: {
type: String,
unique: true
},
createdAt: {
type: Date,
default: Date.now
}
});
// Generate ticket number before saving
bookingSchema.pre('save', async function(next) {
if (!this.ticketNumber) {
this.ticketNumber = `QS${Date.now()}${Math.floor(Math.random() * 1000)}`;
}
next();
});
Explorer
QuickShow1
backend
middleware
auth.js
models
Booking.js
Cinema.js
Event.js
Movie.js
Show.js
User.js
node_modules
routes
admin.js
auth.js
bookings.js
cinemas.js
events.js
movies.js
payments.js
shows.js
uploads
.env
.gitignore
package-lock.json
package.json
seed.js
server.js
frontend
node_modules
package-lock.json
package.json
README.md
SETUP.md
TROUBLESHOOTING.md
Outline
Timeline
Application Builder


QuickShow1 - server.js
server.js
QuickShowCommunityChat.js U
MealSelection.js U
Wishlist.js U
SeatSelection.js U
HeroSlider.js U
MovieDetail.js U
MovieCard.js U
AdminMovies.js U
App.js U
Home.js U
backend > server.js > ...
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const http = require('http');
const socketIo = require('socket.io');
require('dotenv').config();
const User = require('./models/User');
const app = express();
const server = http.createServer(app);
const io = socketIo(server, {
cors: {
origin: process.env.FRONTEND_URL || "http://localhost:3000",
methods: ["GET", "POST"]
}
});
// Middleware
app.use(cors({
origin: ['http://localhost:3000', 'http://127.0.0.1:3000'],
credentials: true,
methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
allowedHeaders: ['Content-Type', 'Authorization']
}));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
// Static files for uploads
app.use('/uploads', express.static('uploads'));
// Socket.io for real-time seat locking and community chat
const seatLocks = new Map(); // showId-seatId -> { userId, timestamp }
io.on('connection', (socket) => {
console.log('User connected:', socket.id);
// Seat locking events
socket.on('lock-seat', ({ showId, seatId, userId }) => {
const lockKey = `${showId}-${seatId}`;
const existingLock = seatLocks.get(lockKey);
if (!existingLock || Date.now() - existingLock.timestamp > 300000) { // 5 min expiry
seatLocks.set(lockKey, { userId, timestamp: Date.now() });
socket.join(`show-${showId}`);
io.to(`show-${showId}`).emit('seat-locked', { showId, seatId, userId });
}
});
socket.on('unlock-seat', ({ showId, seatId }) => {
const lockKey = `${showId}-${seatId}`;
seatLocks.delete(lockKey);
socket.to(`show-${showId}`).emit('seat-unlocked', { showId, seatId });
});
// Community chat events
socket.on('join-community', ({ name }) => {
const safeName = name || 'Guest';
socket.join('community');
io.to('community').emit('community-system', {
name: 'QuickShow',
text: `${safeName} joined the community chat`,
createdAt: new Date().toISOString(),
});
});
Explorer
QuickShow1
backend
middleware
models
node_modules
routes
uploads
.env
.gitignore
package-lock.json
package.json
seed.js
server.js
frontend
node_modules
package-lock.json
package.json
README.md
SETUP.md
TROUBLESHOOTING.md
Outline
Timeline
Application Builder




```
const mongoose = require('mongoose');
const Movie = require('./models/Movie');
const Cinema = require('./models/Cinema');
const Show = require('./models/Show');
const Event = require('./models/Event');
require('dotenv').config();

const sampleMovies = [
  {
    title: "Avengers: Endgame",
    description: "After the devastating events of Avengers: Infinity War, the universe is in ruins. With the help of remaining allies, the Avengers assemble once more in order to reverse Thanos' actions and restore balance to
    genres: ["Action", "Adventure", "Sci-Fi"],
    languages: ["English", "Hindi"],
    cast: [
      { name: "Robert Downey Jr.", role: "Tony Stark / Iron Man", image: "" },
      { name: "Chris Evans", role: "Steve Rogers / Captain America", image: "" },
      { name: "Mark Ruffalo", role: "Bruce Banner / Hulk", image: "" },
      { name: "Chris Hemsworth", role: "Thor", image: "" }
    ],
    crew: [
      { name: "Anthony Russo", role: "Director", image: "" },
      { name: "Joe Russo", role: "Director", image: "" },
      { name: "Christopher Markus", role: "Screenplay", image: "" }
    ],
    duration: 181,
    poster: "https://image.tmdb.org/t/p/w500/or06FN3Dka5tukK1e9sl16pB3iy.jpg",
    banner: "https://image.tmdb.org/t/p/w1280/7RyHsO4yDXtBv1zUU3mTpHeQ0d5.jpg",
    trailerUrl: "https://www.youtube.com/watch?v=TdMBFSGVi1c",
    releaseDate: new Date("2019-04-26"),
    rating: 8.4,
    isNowShowing: true,
    isComingSoon: false
  },
  {
    title: "Spider-Man: No Way Home",
    description: "Peter Parker's secret identity is revealed to the entire world. Desperate for help, Peter turns to Doctor Strange to make the world forget that he is Spider-Man.",
    genres: ["Action", "Adventure", "Fantasy"],
    languages: ["English", "Hindi"],
    cast: [
      { name: "Tom Holland", role: "Peter Parker / Spider-Man", image: "" },
      { name: "Zendaya", role: "MJ", image: "" },
      { name: "Benedict Cumberbatch", role: "Doctor Strange", image: "" }
    ],
    crew: [
      { name: "Jon Watts", role: "Director", image: "" }
    ],
    duration: 148,
    poster: "https://image.tmdb.org/t/p/w500/1gKdhYtq4irTY1GPXvft6k4YLjm.jpg",
    banner: "https://image.tmdb.org/t/p/w1280/14QbnygCuTO0vl7CAFmPf1fgZfV.jpg",
    trailerUrl: "https://www.youtube.com/watch?v=JfVOs4VSpmA",
    releaseDate: new Date("2021-12-17"),
    rating: 8.3,
    isNowShowing: true,
    isComingSoon: false
  },
  {
    title: "Dune",
    description: "Paul Atreides leads a rebellion to restore his family's reign over the desert planet Arrakis while grappling with a terrible destiny.",
    genres: ["Sci-Fi", "Adventure", "Drama"],
    languages: ["English"],
    cast: [
      { name: "Timothée Chalamet", role: "Paul Atreides", image: "" },
      { name: "Rebecca Ferguson", role: "Lady Jessica", image: "" },
      { name: "Oscar Isaac", role: "Duke Leto Atreides", image: "" }
    ],
```

QuickShow1 - admin.js

server.js | admin.js | QuickShowCommunityChat.js U | MealSelection.js U | Wishlist.js U | SeatSelection.js U | HeroSlider.js U | MovieDetail.js U | MovieCard.js U | AdminMovies.js U | App.js U

backend > routes > admin.js > ...

```
1  const express = require('express');
2  const { protect, admin } = require('../middleware/auth');
3  const Movie = require('../models/Movie');
4  const Cinema = require('../models/Cinema');
5  const Show = require('../models/Show');
6  const Booking = require('../models/Booking');
7  const User = require('../models/User');
8  const multer = require('multer');
9  const path = require('path');
10
11 const router = express.Router();
12
13 const storage = multer.diskStorage({
14   destination: (req, file, cb) => {
15     cb(null, 'uploads/');
16   },
17   filename: (req, file, cb) => {
18     cb(null, `${Date.now()}-${file.originalname}`);
19   }
20 });
21
22 const upload = multer({ storage });
23
24 // ======== MOVIES MANAGEMENT ========
25
26 // @route   POST /api/admin/movies
27 // @desc    Create movie
28 router.post('/movies', protect, admin, upload.fields([
29   { name: 'poster', maxCount: 1 },
30   { name: 'banner', maxCount: 1 }
31 ]), async (req, res) => {
32   try {
33     const movieData = req.body;
34     if (req.files?.poster) {
35       movieData.poster = `/uploads/${req.files.poster[0].filename}`;
36     }
37     if (req.files?.banner) {
38       movieData.banner = `/uploads/${req.files.banner[0].filename}`;
39     }
40
41     const movie = await Movie.create(movieData);
42     res.status(201).json({ success: true, movie });
43   } catch (error) {
44     res.status(500).json({ message: 'Server error', error: error.message });
45   }
46 });
47
48 // @route   PUT /api/admin/movies/:id
49 // @desc    Update movie
50 router.put('/movies/:id', protect, admin, upload.fields([
51   { name: 'poster', maxCount: 1 },
52   { name: 'banner', maxCount: 1 }
53 ]), async (req, res) => {
54   try {
55     const movieData = req.body;
56     if (req.files?.poster) {
57       movieData.poster = `/uploads/${req.files.poster[0].filename}`;
58     }
59     if (req.files?.banner) {
60       movieData.banner = `/uploads/${req.files.banner[0].filename}`;
61     }
62
63     const movie = await Movie.findByIdAndUpdate(req.params.id, movieData, { new: true });
64     res.json({ success: true, movie });
65   } catch (error) {
```

Explorer
QuickShow1
backend
middleware
models
node_modules
routes
admin.js
auth.js
bookings.js
cinemas.js
events.js
movies.js
payments.js
shows.js
uploads
.env
.gitignore
package-lock.json
package.json
seed.js
server.js
frontend
node_modules
package-lock.json
package.json
README.md
SETUP.md
TROUBLESHOOTING.md
Outline
Timeline
Application Builder

main* | Compile Project | AWS | BLACKBOX Chat | Add Logs | CyberCoder | Improve Code | Share Code Link | Open Website | Ln 1, Col 1 | Spaces: 2 | UTF-8 | LF | JavaScript | Go Live | BLACKBOXAI: Open Chat | Free - Upgrade Now | Windsurf - Settings | Prettier

QuickShow1 - payments.js

backend > routes > payments.js

```javascript
const express = require('express');
const axios = require('axios');
const { protect } = require('../middleware/auth');
const Booking = require('../models/Booking');

const router = express.Router();

const PAYMENT_API_URL = process.env.PAYMENT_API_URL || 'http://localhost:3100';
const PAYMENT_API_VERSION = process.env.PAYMENT_API_VERSION || 'v68';
const MERCHANT_ACCOUNT = process.env.PAYMENT_MERCHANT || 'QuickShowMockAccount';

// @route   POST /api/payments/create-order
// @desc    Create payment session with mock Adyen API
router.post('/create-order', protect, async (req, res) => {
  try {
    const { bookingId, paymentMethodType = 'scheme' } = req.body;

    const booking = await Booking.findById(bookingId);
    if (!booking) {
      return res.status(404).json({ message: 'Booking not found' });
    }

    if (booking.userId.toString() !== req.user._id.toString()) {
      return res.status(403).json({ message: 'Access denied' });
    }

    const payload = {
      amount: {
        currency: 'INR',
        value: booking.amountPaid
      },
      reference: booking.ticketNumber,
      merchantAccount: MERCHANT_ACCOUNT,
      paymentMethod: {
        type: paymentMethodType
      },
      shopperEmail: req.user.email,
      storePaymentMethod: false,
      returnUrl: process.env.PAYMENT_RETURN_URL || 'http://localhost:3000/payment/success',
      additionalData: {
        bookingId: booking._id.toString(),
        userId: req.user._id.toString()
      }
    };

    const paymentResponse = await axios.post(
      `${PAYMENT_API_URL.replace(/\/$/, '')}/${PAYMENT_API_VERSION}/payments`,
      payload,
      {
        headers: {
          'Content-Type': 'application/json'
        }
      }
    );

    const paymentResult = paymentResponse.data || {};
    const normalizedResultCode = (paymentResult.resultCode || 'Authorised').toLowerCase();
    const failureStatuses = ['refused', 'cancelled', 'failed'];
    const isApproved = !failureStatuses.includes(normalizedResultCode);

    booking.paymentId = paymentResult.pspReference || booking.paymentId || `mock_payment_${Date.now()}`;
    booking.razorpayOrderId =
      paymentResult.merchantReference ||
      paymentResult?.order?.pspReference ||
      booking.razorpayOrderId ||
```

Bibliography & References

1. Websites and Official Documentation

- MDN Web Docs – HTML, CSS, and JavaScript reference
- React Official Documentation
- Node.js and Express.js Documentation
- MongoDB Official Documentation
- Razorpay API Documentation

2. Tutorials and Learning Platforms

- GeeksforGeeks – Web development tutorials
- W3Schools – Programming references
- freeCodeCamp – Full-stack development tutorials

3. Development Tools and Platforms

- **Visual Studio Code –** Source code editor
- **GitHub –** Code hosting and collaboration
- **Postman –** API development and testing
- **Figma –** UI/UX design and prototyping

4. Industry References

- **BookMyShow –** Feature and UI reference
- **IMDb –** Movie-related data reference